Probing Reinforcement-Learned Representations for Geo-Free Guard Placement in the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija[a,*], Jurica Maltar[a], Nathan Chappel, Domagoj Matijević[a]

[a]*School of Applied Mathematics and Informatics, University J. J. Strossmayer of Osijek, Croatia*

Abstract

Reinforcement learning can train models to solve hard combinatorial problems from raw inputs, but what such a model has learned, as opposed to what it outputs, is hard to establish. We study this on the vertex-guard Art Gallery Problem under a constraint we call *geo-free* inference: at test time the model sees only vertex coordinates, never a visibility computation, though training and evaluation use visibility freely. A pointer-network policy trained by a coverage-aware reward places guards economically but leaves a tail of under-covered polygons that grows beyond the training sizes. Freezing its encoder and reading the embeddings with a small classifier removes most of that tail at a cost in guards, cutting under-covered polygons several-fold in distribution and about ten-fold on instances five times the training size. Compared at equal guard count, the encoder rather than the classifier's own capacity accounts for the gain, and this holds on four independently trained policies; hand-designed geometric features recover much of the encoder's advantage, yet the frozen features still carry guard-relevant structure those features miss, and vice versa. We conclude that the reinforcement-trained encoder holds much of the guard-placement structure its decoder failed to express, tied to the conventions of the instances it was trained on. The method measures what such a model has learned; it is not a competitive Art Gallery solver.

Keywords: Art Gallery Problem, neural combinatorial optimization, reinforcement learning, pointer networks, representation probing

1. Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [1, 2], with applications in

*Corresponding author

Email addresses: dseverdi@mathos.hr (Domagoj Ševerdija), jmaltar@mathos.hr (Jurica Maltar), nathan.s.chappel@gmail.com (Nathan Chappel), domagoj@mathos.hr (Domagoj Matijević)

surveillance, sensor placement, and robotics [3, 4]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon's vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We refer to inference under this policy as *geo-free*: the network reads only vertex coordinates at the moment it places guards, never querying a visibility oracle (defined precisely in Section 2.3). We are not asking whether a learned solver can match a classical greedy heuristic on guard count; it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

Why geo-free inference. One could object that the coordinates are given, so a solver could just compute visibility from them; a visibility-based greedy does exactly that, at a lower guard count. The constraint withholds the computation, not the information: greedy evaluates visibility at every step, while the geo-free model must have learned it. It is therefore a measurement instrument rather than a deployment recipe, and where an exact oracle is cheap to query a classical solver is simply the better tool. Section 2.3 makes the argument precise.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [5] on Bradley–Terry preference pairs [6] drawn from the policy's own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [7, 8] with binary preferences between pairs of policy rollouts, preserving a usable gradient signal even when the reward saturates under the coverage constraint [5].

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the 0.95 coverage gate one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, offers no coverage guarantee.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier called *probe* conditioned on the policy's frozen encoder embeddings (together with the vertex coordinates and a binary indicator of the policy's own guard set, all geo-free), and predict a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the coverage tail substantially both in and out of distribution. A no-encoder ablation that zeroes the embeddings while keeping the coordinates and seed indicator (Section 6.2), together with a probe-capacity ladder that holds the encoder fixed while shrinking the probe,

isolates the encoder's contribution from the other inputs and from the probe's own capacity. Those comparisons have to be made at *matched guard budget* rather than at a common threshold, because removing an input also changes how many vertices the probe selects and extra guards raise coverage on their own; at a fixed threshold the ordering can invert. Read that way, we take the gap as evidence that the reinforcement-trained encoder already carries guard-relevant structure a small probe can turn into a gate-clearing decision, so the policy's coverage tail reflects, on the better-supported reading, what its decoder expressed rather than a limit of the representation. The cost of clearing the tail is a controlled rise in guard count, which we report explicitly.

Contributions. This paper makes four contributions.

C1 A representation–decoder decomposition for neural combinatorial optimization. We adapt the probing methodology of representation analysis [9, 10, 11] to a reinforcement-trained combinatorial solver: freezing the policy's encoder and training a single-shot per-vertex probe over its embeddings (with the vertex coordinates and the policy's seed) isolates what the learner *represents* from what its decoder *expresses*, with a 2×2 encoder–seed ablation, a probe-capacity ladder (linear, attention-free MLP, full probe), and an analytic-geometry feature baseline as controls. Concurrent work has begun probing neural *routing* solvers [12, 13]; to our knowledge ours is the first probing study of an RL-trained policy on a *constrained geometric covering* problem, and the first to read a coverage decision (per-vertex guard membership) from a frozen encoder under a geo-free inference constraint.

C2 Out-of-distribution generalization at scale, replicated across policies. On a large held-out set spanning the training size range and extending to roughly five times beyond it, the probe raises the average fraction clearing the 0.95 coverage gate across four probe seeds from 85% (policy seed) to 99%; the gain holds on the genuinely larger-than-training polygons alone (85% $\rightarrow$ 99.2%, Section 6.3). It is not a property of one training run: three further policies trained from scratch under the same recipe show the same tail-closing effect out of distribution, and the encoder's matched-budget advantage holds on all four in distribution (Section 6.4).

C3 Geo-free inference as a protocol. Restricting test-time inputs to vertex coordinates and learned features derived from them, never a visibility oracle, isolates what a learner internalizes from what such an oracle could compute for it, making "no explicit geometric knowledge" a measurable property rather than a rhetorical one (Section 2.3).

C4 A structural finding on post-processing. Learned iterative editors fail to improve the policy's seed without dropping coverage (error accumulation, stop-time miscalibration, and train/inference drift), whereas the single-shot probe avoids all three by design and is empirically stable

under iteration: re-running it on its own output leaves coverage essentially unchanged across the threshold range, with the guard count drifting slowly, up at the operating thresholds and down into a contraction as the threshold approaches the decision boundary. We measure this over the whole range $t \in [0.2, 0.8]$ (Section 6.5). We read this as support for the single-shot design as a stable reading of the encoder; the fixed point is an empirical property, not a proof that the probe is a clean representation readout.

The components we use, pointer networks, Bradley–Terry preference optimization, frozen-representation probing, and a Transformer per-vertex classifier, are all established. Our contribution is their combination and adaptation to a constrained geometric covering problem, and the representation–decoder measurement it enables.

The remainder of the paper is organized as follows. Section 2 establishes notations and definitions. In Section 3 related work is reviewed. Section 4 describes the reinforcement method and the representation probe. Finally, Section 5 describes the experimental setup, Section 6 reports the results, followed by discussion in Section 7, limitations in Section 8, and the conclusion in Section 9.

2. Problem and Notation

2.1. Polygons and visibility

Let $P \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \ldots, x_n \in \mathbb{R}^2$ and vertex index set $V(P) = \{1, \ldots, n\}$. For two points $p, q \in P$ we say that p *sees* q when the closed segment between them stays inside the polygon, $\overline{pq} \subseteq P$. Guards are placed at vertices: for a vertex-guard set $S \subseteq V(P)$, the visibility region and area-normalized coverage are

$$\mathrm{Vis}_P(S) \;=\; \{\, q \in P : \exists\, i \in S,\; x_i \text{ sees } q \,\}, \qquad \mathrm{Cov}_P(S) \;=\; \frac{\mathrm{Area}(\mathrm{Vis}_P(S))}{\mathrm{Area}(P)} \in [0,1]. \tag{1}$$

2.2. The vertex-guard variant

The *vertex-guard Art Gallery Problem (AGPVG)* asks for the smallest vertex-guard set that covers the whole polygon. We define the *vertex-guard optimum* directly:

$$\mathrm{OPT}(P) \;=\; \min\big\{|S| : S \subseteq V(P),\; \mathrm{Cov}_P(S) = 1\big\}. \tag{2}$$

Restricting guards to $V(P)$ reduces placement to a finite discrete choice over n vertices, the natural action space for a pointer-network policy [14, 8, 15, 16], while retaining the full NP-hardness and non-uniform visibility structure of the problem.

Note that equation (2) is a *constrained* single-objective problem (minimize the guard count subject to the hard constraint $\mathrm{Cov}_P(S) = 1$) and a discrete set-cover instance in which coverage is the submodular union of per-guard visibility

regions. This is what separates it from the routing problems on which neural combinatorial solvers are usually studied, where any permutation is feasible and only a single scalar cost is minimized. A learner denied a visibility oracle at inference (the geo-free constraint, Section 2.3) cannot certify the coverage constraint while placing guards; it must instead trade coverage against guard count, so the single constrained objective becomes a two-axis operating curve.

2.3. Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon's vertex coordinates and learned features derived from them: no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step, as classical local search and active-search decoding both do, can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes "no explicit geometric knowledge" a measurable property.

3. Related Work

This section places the present work in four lines of research: algorithms for the Art Gallery Problem (classical and learned), neural combinatorial optimization with reinforcement learning, supervised post-processing of learned solvers, and probing of learned representations.

The Art Gallery Problem. Lee and Lin [1] established NP-hardness for several variants of the AGP; O'Rourke [2] surveys the foundational geometric and combinatorial results. Work on the problem has traditionally followed three lines. Exact algorithms solve smaller instances to optimality, most effectively via integer programming: Couto et al. [17] compute exact vertex-guard optima this way, and we treat their published instance library [18], with its exact optima, as the source of polygons in our experiments. Approximation algorithms provide provable guarantees: Ghosh [19] gives a logarithmic-factor method for guarding simple polygons. Heuristics without theoretical guarantees, of which the classical greedy rule [20] that places guards to maximize marginal coverage is the most common, are reliable on small to mid-sized polygons. Because area coverage is a monotone submodular function of the guard set, this marginal-coverage rule is exactly the greedy submodular-maximization heuristic and inherits its diminishing-returns approximation guarantee [21]; the guarantee is available only to a solver that can evaluate marginal coverage while it selects, which a geo-free learner cannot (Section 2.3), so we forgo such guarantees by construction, the price of measuring what the representation has learned rather than

what an oracle computes. The same exact/approximation/heuristic split recurs across the problem's many variants; for instance, guarding 1.5-dimensional terrains admits constant-factor approximations [22, 23]. Local-search refinement of a candidate guard set by swapping or removing vertices is the standard high-quality post-processing step. We use classical greedy purely as a non-learned evaluation reference, not in competition with the reinforcement method on guard count.

Reinforcement learning has been applied to the AGP and to closely related coverage problems before, but in settings quite different from ours. Liao et al. [24] discretize the polygon into a grid and train a per-instance tabular Q-learning agent that trades guard count against covered area; Kaba et al. [25] cast the 3D view-planning problem, a sensor-coverage set-cover, as an MDP solved with value-based RL (SARSA, Q-learning, temporal-difference) guided by a geometry-aware score, outperforming the greedy baseline. Both methods consult coverage or visibility while placing guards and learn a fresh policy for each instance. We differ on three axes: the action space is the polygon's own vertices rather than grid cells or candidate viewpoints; a single amortized policy generalizes to unseen polygons without retraining; and inference is geo-free, with no visibility computation at the moment guards are placed.

Reinforcement learning for combinatorial optimization. Pointer networks [14] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [8] using REINFORCE [7] with a learned baseline; Deudon et al. [26] similarly train TSP construction heuristics by policy gradient. Subsequent work introduced attention-only architectures for routing problems [15], policy-optimization variants such as POMO [16] that exploit problem symmetries, and Transformer pointer policies such as Pointerformer [27] aimed at generalizing to large instances, a concern we share, since our evaluation stresses out-of-distribution polygon sizes. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [5], which trains the policy with a Bradley–Terry [6] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [28].

A recurring feature of this literature is worth making explicit, because AGP departs from it. Routing problems such as the TSP carry a single scalar objective and free feasibility (any permutation is a valid tour), and even constrained variants such as capacitated routing usually enforce feasibility by masking infeasible actions at decode time, which presumes an oracle for the constraint. Vertex-guard AGP has neither property: it is a constrained set-cover problem (Section 2.2), and under geo-free inference the coverage constraint cannot be masked in, so it can only be learned and traded off against guard count, the

operating curve our threshold sweep traces. The closest learned analogues are RL methods that approximate Pareto frontiers for bi-objective routing [29, 30]; but those balance two *soft* costs, whereas one of our two axes is a hard feasibility constraint, making PA-Net's Lagrangian relaxation of a constraint [29] a closer match than a symmetric Pareto front. On the optimization side, Caramanis et al. [31] prove that policy-gradient solution-samplers enjoy benign landscapes for a broad class of problems (Max-/Min-Cut, Max-k-CSP, bipartite matching, TSP), but their guarantee assumes a single cost that is *bilinear* in instance and solution features and imposes no hard feasibility constraint. AGP's guard-count term is linear and would fit, but its coverage term is the submodular union of visibility regions and its full-coverage requirement is a hard, geo-free-uncertifiable constraint, placing AGP outside their analyzed class and helping explain why a naive policy gradient struggles here (Section 4.1).

Supervised post-processing of learned solvers. An orthogonal line of work post-processes a base solver's output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [32] and its variants partially address the latter. We use a single-shot supervised post-processor (the SetPredictor) not as a competing solver but as a *probe* of the RL-trained encoder's representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover the coverage the decoder left short, the RL encoder must already carry the relevant geometric information. Directly cloning search solutions with a recurrent decoder, by contrast, fails on this problem for a structural reason, the teacher-forcing cascade we return to in Section 6.5, which is why we read the frozen encoder with a single-shot, recurrence-free probe rather than a sequential supervised decoder.

Probing learned representations. The methodology of freezing a trained network and reading its internal representations with a small supervised classifier originates in the analysis of vision and language models: linear classifier probes diagnose what intermediate layers encode [9], control tasks calibrate how much of a probe's accuracy reflects the representation rather than the probe's own capacity [10], and Belinkov [11] surveys the family. A central methodological caveat runs through this literature: a sufficiently expressive probe can achieve high accuracy by *computing* the target itself rather than reading it off the representation, so probing results must be interpreted against a capacity control [10]. We take this concern seriously below by reporting a capacity ladder (a zero-hidden-layer linear probe, an attention-free MLP, and the full SetPredictor) alongside a no-encoder control task, so that the effect we attribute to the representation is the part that survives holding probe capacity fixed and removing the encoder.

Probing has only recently reached neural combinatorial optimization, and the closest work is concurrent. Zhang et al. [12] freeze trained routing models (AM, POMO, LEHD) on the TSP and CVRP and read their embeddings with

linear probes for a low-level geometric quantity (pairwise Euclidean distance), for solution-structure signals (whether a link avoids a myopic greedy choice; whether two nodes lie on the same route), and for a capacity-constraint signal, with a coefficient-significance tool (CS-Probing) that localizes which embedding dimensions carry each. Narad et al. [13] similarly probe TSP embeddings for node-removal and edge-forbid sensitivity. We use the same freeze-and-probe method but probe something different in a different setting. Our problem is covering, not routing: under geo-free inference the coverage constraint cannot be enforced by decode-time masking, so the model must learn it and trade it against guard count. Our target is global, per-vertex membership in a guard set whose visibility regions must cover the polygon, rather than a pairwise quantity like inter-node distance; visibility depends on the whole boundary, so recovering it from the embedding says more about learned geometry than recovering a distance does. The geo-free limit on what the probe may read at inference has no counterpart in the routing studies. And where they probe linearly, avoiding the capacity question but seeing only linear signal, our probe is an expressive Transformer that also serves as the post-processor, so we run a capacity ladder (linear, attention-free MLP, full Transformer; Section 6.2) to confirm the signal comes from the representation and not the probe. The learning frameworks differ only in recipe, our Bradley–Terry preference optimization versus their policy-gradient (AM, POMO) or supervised (LEHD) training, but since AM and POMO are also RL-trained, this is a difference in algorithm, not in paradigm, and we do not rely on it to distinguish our work. Our no-encoder ablation is the control task [10]: it holds probe capacity and training fixed while removing the representation, so the gap is attributable to the encoder.

4. Method

We describe the reinforcement method (Section 4.1) and the single-shot representation probe (Section 4.2). The empirical diagnostic that most motivates the single-shot design, i.e., why iterative editors fail to improve the seed, is deferred to the results (Section 6.5).

4.1. The reinforcement method: PO/BT pointer policy

Architecture. Figure 1 shows the full pipeline and where the geo-free boundary falls. The policy is a compact pointer network with a single-layer, unidirectional LSTM [33] encoder and an attention-based decoder. A polygon P with vertex coordinates $(x_1, \ldots, x_n)$ is consumed coordinate-wise (no visibility input). Running the LSTM over the vertex sequence yields per-vertex hidden states that serve as the embeddings $(h_1, \ldots, h_n)$, $h_i \in \mathbb{R}^d$, where d is the encoder embedding dimension (its value, and those of all dimensions introduced below, is set in Section 5.2). The decoder attends over the encoder at each step, emits a vertex index or an *end-of-sequence* (EOS) token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \ldots, \pi_{|\pi|})$ while the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$. All metrics, i.e., coverage,

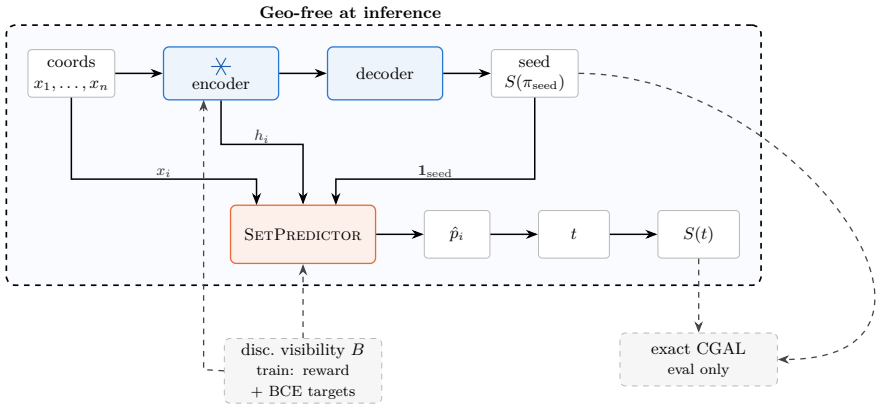

Figure 1: The full inference pipeline. The frozen LSTM encoder (trained by PO/BT, Section 4.1) turns vertex coordinates into per-vertex embeddings h_i; its own decoder greedily decodes a seed guard set $S(\pi_{\text{seed}})$. The SetPredictor (Section 4.2) reads the frozen embeddings, the coordinates, and the seed indicator (Eq. (7)) and outputs a per-vertex probability $\hat{p}_i$, thresholded at t into the final guard set $S(t)$. Everything inside the dashed boundary is geo-free: no visibility object is read at the moment guards are placed (Section 2.3). Discretized visibility B (Eq. (4)) shapes the PO/BT reward and the SetPredictor's BCE targets $S^{\star}_{\text{LS}}$ during training only; exact CGAL scores both guard sets only at evaluation (Section 5.3), both cross the geo-free boundary from outside it, never from within.

$|S|/n$, $|S|/\text{OPT}$, are computed from $S(\pi)$. We denote the joint encoder–decoder distribution by $p_\theta(\pi \mid P)$, where θ collects all policy parameters.

The encoder has no training signal of its own: its weights change only because the reward gradient back-propagates from the decoder's rollouts. If the encoder already holds the geometric information needed for feasibility, a small classifier reading its frozen embeddings should be able to recover the guard set the decoder failed to express. This is why we probe the encoder inside the trained policy by freezing it and attaching a small classifier. By contrast, a standalone model, i.e., an encoder + a probe trained from scratch, would reflect its own learning, not what the reinforcement policy internalized. The policy's greedy decode additionally supplies the seed guard set that the representation probe (Section 4.2) later refines.

Why an autoregressive pointer. Vertex-guard placement is a set-cover problem: whether a vertex should be a guard depends on what the guards already chosen leave uncovered. An autoregressive pointer captures this dependency naturally by conditioning each pick on the partial solution (Eq. (5)), and it decides cardinality through the EOS token rather than a fixed threshold. A model scoring vertices independently would miss this structure, which is also why the probe (Section 4.2) is not a standalone solver: unable to be autoregressive, it takes the policy's greedy seed as an input instead.

The reason the decoder is necessary once the probe works, i.e., why we cannot discard it, is that the encoder's representation was shaped entirely by the reward

gradient flowing through the decoder, and the probe reads the decoder's seed as a feature. Removing the decoder removes both the representation and the probe's input. What we compare is encoder-and-decoder against encoder-and-probe, not the decoder against no-decoder. Reading the frozen encoder with a probe also recovers coverage on polygons far larger than those seen in training (Section 6.3), which autoregressive pointer policies alone cannot do [14, 27].

Reward and rollouts. For each polygon P we sample R rollouts from the policy and score each rollout's guard set $S(\pi)$ by a scalar utility that gates coverage at τ and then prefers smaller guard sets,

$$u(\pi \mid P) \;=\; \min\big(\mathrm{Cov}_P(S(\pi)),\, \tau\big) \;-\; \lambda\, \frac{|S(\pi)|}{|V(P)|} \;-\; \rho\, \max\big(0,\, \tau - \mathrm{Cov}_P(S(\pi))\big), \tag{3}$$

where τ is the coverage gate, λ weights guard usage, and ρ penalizes coverage below the gate (concrete values given in Section 5.2). Capping the coverage term at τ removes any incentive to over-cover, so once a rollout meets the coverage gate the utility is driven purely by cardinality. Coverage during training is computed with the discretized-visibility sampler described next as a cheap approximation of the exact coverage area.

Discretized-visibility coverage. Computing the exact coverage of a guard set (the area of the union of its vertices' visibility polygons) is too costly to repeat for the many rollouts and local-search candidates scored per polygon, so during training we estimate it by Monte-Carlo sampling. For each polygon we draw M points uniformly from its interior (rejection sampling within the bounding box, under a fixed per-polygon seed) and, for each vertex i, compute its *exact* visibility polygon $\mathrm{Vis}_P(\{i\})$ once with CGAL triangular-expansion visibility [34]. Recording which samples fall inside each visibility polygon yields a boolean visibility matrix $B \in \{0,1\}^{|V(P)|\times M}$ with $B_{is} = 1$ iff sample s is visible from vertex i, and the coverage of a guard set S is estimated by the covered-sample fraction

$$\widehat{\mathrm{Cov}_P(S)} \;=\; \frac{1}{M}\,\Big|\big\{\, s : \exists\, i \in S,\; B_{is} = 1 \,\big\}\Big|, \tag{4}$$

a uniform Monte-Carlo estimate of the area ratio $\mathrm{Cov}_P(S)$ of Eq. (1), evaluated as a bitwise OR over the rows of B indexed by S. Because B depends only on the polygon, it is computed once and cached, so every subsequent coverage query during a rollout or a local-search edit costs an $O(|S|\,M)$ bitwise operation rather than a polygon-union computation. The per-vertex visibility polygons are exact; the only approximation is replacing the area integral by a sample average, each sample is an independent Bernoulli trial with success probability $\mathrm{Cov}_P(S)$, so the estimate is a binomial proportion whose standard error is the familiar $\sqrt{\mathrm{Cov}_P(S)(1-\mathrm{Cov}_P(S))/M} = O(1/\sqrt{M})$. Exact coverage (the area of the union, computed from the polygon geometry rather than sampled) is reserved for evaluation (Section 5.3).

Preference-optimization loss. We score a rollout by its *length-normalized* log-likelihood, the mean per-token log-probability under the factorization

$$p_\theta(\pi \mid P) = \prod_t p_\theta(\pi_t \mid \pi_{<t}, P),$$

$$\bar{\ell}_\theta(\pi \mid P) \;=\; \frac{1}{|\pi|} \sum_{t=1}^{|\pi|} \log p_\theta(\pi_t \mid \pi_{<t}, P), \tag{5}$$

which keeps the preference from being biased by guard-set size, since AGP solutions have variable length (cf. Pan et al. [5]). For each pair (π^+, π^-) of rollouts with $u(\pi^+ \mid P) > u(\pi^- \mid P)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\mathrm{BT}}(\theta) \;=\; -\mathbb{E}_{(P,\pi^+,\pi^-)} \left[\log \sigma\big(\alpha \left[\bar{\ell}_\theta(\pi^+ \mid P) - \bar{\ell}_\theta(\pi^- \mid P) \right]\big) \right], \tag{6}$$

where $\alpha > 0$ is a temperature scaling the preference margin (details in Section 5.2). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

The choice of PO over REINFORCE follows Pan et al. [5]. Once a rollout meets the coverage constraint, the reward of Eq. (3) flattens and the policy-gradient signal on guard count vanishes, a saturation effect specific to hard coverage constraints, not covered by benign-landscape guarantees for unconstrained problems [31]. The choice is therefore by design rather than by tuning. On initial development runs, REINFORCE variants (a learned-critic baseline and a greedy-rollout baseline) over-guarded substantially, and PO/BT gave better guard counts at comparable coverage, so we adopted it and did not pursue the REINFORCE baselines further. Bradley–Terry preferences between rollout pairs remain informative even when both rollouts saturate, since the signal depends on which rollout is relatively better, not on the absolute reward level.

At the end of training the encoder is frozen and treated as a fixed feature extractor for the rest of the paper.

4.2. Probing the representation: a single-shot per-vertex classifier

The policy learns to decrease the cardinality but leaves a coverage tail (Section 6.1). We first verified that this tail is *reachable*: running a local-search editor (add/remove/swap) on the policy's own greedy seed lifts every below-gate in-distribution polygon back over the 0.95 gate while *reducing* the guard count (the *LS on policy seed* row of Table 2). We treat this as a sanity check on *reachability*, not as a diagnosis of cause: it shows the missing coverage is a handful of local edits away from the seed, which is consistent with a decoder that stopped early, and, on the other hand, equally possible under an encoder that under-represents. Distinguishing the two is what the encoder ablation is for (Section 6.2). As a by-product the editor yields a concrete target set $S^\star_{\mathrm{LS}}$. The open question is then whether that refinement can be *learned* from the frozen encoder, geo-free. A learned *iterative* editor over the seed cannot: it does

not close the tail in any coverage-preserving regime, contrary to the fine-tuning arguments of [5], for reasons we diagnose in Section 6.5. We therefore probe what the policy has learned with a *single-shot, rollout-free* classifier that recovers the coverage decision the decoder failed to make. Its inputs are geo-free (the frozen encoder embedding, the vertex coordinates, and the policy's seed indicator), with no visibility object at inference. We call it the SetPredictor and present it primarily as a representation diagnostic.

For polygon P with vertex coordinates $x_1, \ldots, x_n$, the input feature for vertex i is

$$z_i \;=\; [\, h_i \,;\, x_i \,;\, \mathbf{1}\{i \in S(\pi_{\text{seed}})\}\,] \;\in\; \mathbb{R}^{d+3}, \tag{7}$$

that is, the *frozen* d-dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i, and a single binary indicator for whether vertex i is selected by the policy's greedy-decoded seed π_{seed}.

The architecture is a small Transformer encoder (parameters ϕ) over the n vertex tokens with a sigmoid output head per token (sizes given in Section 5.2). The features z_i are linearly projected to width d and passed through L pre-norm self-attention blocks (each with H-head self-attention at width d, a GELU feed-forward sublayer of expansion factor f, and residual connections), followed by a final layer normalization; we write v_i for the resulting per-vertex hidden state. This v_i is the SetPredictor's *own* encoding of vertex i, and is distinct from the frozen policy embedding h_i, which enters only as part of the input feature z_i in Eq. (7). From the $(v_1, \ldots, v_n)$ we build two context vectors by mean-pooling: a *seed* context $c_{\text{seed}} = \frac{1}{|S(\pi_{\text{seed}})|} \sum_{i \in S(\pi_{\text{seed}})} v_i$, averaged over the vertices the policy placed in its seed, and a *global* context $c_{\text{glob}} = \frac{1}{n} \sum_{i=1}^{n} v_i$, averaged over all vertices of the polygon. Both are single vectors shared across vertices. Each per-vertex prediction concatenates the vertex's own v_i with these two context vectors and passes the result through a 2-layer GELU MLP head ($3d \to d \to 1$):

$$\hat{p}_i \;=\; \sigma\big(\mathrm{MLP}([\, v_i \,\|\, c_{\text{seed}} \,\|\, c_{\text{glob}} \,])\big), \qquad i = 1, \ldots, n. \tag{8}$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) \;=\; \{\, i \in \{1, \ldots, n\} : \hat{p}_i \geq t \,\}. \tag{9}$$

Objective. The probe is trained by supervised learning to predict membership in an LS-improved target set $S^{\star}_{\text{LS}}(P)$ (the policy's seed refined by local search, constructed as described in Section 5.2), so it is not itself a reinforcement learner. We supervise on this refinement rather than on the exact optimum for two reasons: guard membership at the optimum is non-unique, since the constraint in Eq. (2) fixes cardinality, not which vertices are chosen (Section 6.5), so an optimal set is an ill-posed per-vertex label, and the optimum is in any case unavailable at scale (Section 5.3); the LS endpoint, by contrast, is a well-defined, coverage-restoring refinement of the policy's *own* seed. With $y_i = \mathbf{1}\{i \in S^{\star}_{\text{LS}}(P)\}$, a positive-class weight w_+ that balances the empirical non-guard-to-guard ratio, and $\mathcal{B}$ a mini-batch of polygons, the per-vertex weighted

binary cross-entropy is

$$\mathcal{L}_{\mathrm{BCE}}(\phi) \;=\; -\frac{1}{\sum_{P\in\mathcal{B}} |V(P)|} \sum_{P\in\mathcal{B}} \sum_{i=1}^{|V(P)|} \big[\, w_{+} y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i) \,\big]. \quad (10)$$

Whether closing that tail is the encoder's doing rather than the probe's own capacity is settled not here but by the controls of Section 6.2: the no-encoder ablation and a zero-capacity linear probe.

Inference and the threshold knob. At inference we run the policy encoder once on the coordinates, greedy-decode the seed π_{seed}, form the features z_i of Eq. (7), run one SetPredictor forward pass to obtain $(\hat{p}_1, \ldots, \hat{p}_n)$, and threshold to return $S(t)$ (Eq. (9)). The policy seed, the local-search editor, and the discretized-visibility scoring are all *training-time* scaffolding used only to build the labels $S^{\star}_{\mathrm{LS}}$; at inference the method is this single geo-free forward pass, no seed-refinement loop, no local search, no visibility object, so LS is part of how the probe is *trained*. The threshold t is the single knob trading coverage for guard count. We report the headline results at $t \in \{0.20, 0.25, 0.30\}$ (Section 6.2) and, additionally, sweep it over the full range $[0.05, 0.80]$ to trace the whole operating curve (Figure 3) and to place ablation conditions at matched guard budget rather than at a common threshold (Table 5). Re-running the probe with its own output as the updated seed indicator changes the prediction negligibly (Section 6.5), so we use a single pass throughout.

5. Experiments

5.1. Dataset and partition

We draw polygons primarily from the *Art Gallery Problem with Vertex Guards* (AGPVG) instance library of Couto, de Rezende, and de Souza [18, 17], which collects several polygon families (random orthogonal (including large-area `fat` and small-area `min` variants), random simple (`randsimple`), random von Koch (`randvon`), complete von Koch (`vonkoch`), and boundary-simple (`simple`)) over vertex counts n ranging from 8 to 2,500. The library distributes vertex-guard optimal solutions for its instances, which we use directly as OPT.

We supplement the library with 313 additional random simple polygons (`random` class, $n \in \{20, 40, \ldots, 600\}$, 30 instances per size) to increase training density for random simple polygons in the in-distribution range; these share the same rational-coordinate format, and their vertex-guard optima were computed by solving the vertex-guard set-cover integer program. The splits we work with are listed in Table 1.[1] The held-out evaluation splits are deduplicated against `train` to remove any instance shared with it (train/eval leakage); every evaluation number in this paper is computed on these leak-free splits, and it is this

[1] The 70/30 `dev/test` partition uses a seeded random shuffle (seed 1234) rather than an alphabetical-name split due to instance naming

deduplication that separates them from the earlier protocol under which the development-time REINFORCE runs of Section 4.1 were scored.

Split	# polygons	n range	Use
train	8,867	8–198	Pointer + SetPredictor training
dev	840	16–198	SetPredictor decision threshold t, start rule
test	362	8–192	Held-out evaluation (in-distribution)
ood	2,081	8–1,000	Out-of-distribution evaluation
ood-large	285	600–2,250	Extreme-OOD evaluation

Table 1: Dataset partition. The `dev` and `test` splits share the same source library files, partitioned 70/30 by seeded random shuffle (seed 1234): `dev` is the hyperparameter split, used to choose the decision threshold and the canonical start rule of Section 8, and `test` is the in-distribution held-out evaluation. Neither model's checkpoint is selected on `dev`: the policy's is selected on `train` and the probe's is simply the final epoch (Section 5.2). The `ood` split is a larger held-out set spanning the training size range and extending well beyond it, to $n = 1000$ (about five times the training maximum); its genuinely larger-than-training polygons ($n > 198$) are analyzed separately in Section 6.3. The `ood-large` split is used for an extreme-OOD evaluation (Table 9).

5.2. Training setup

The PO/BT policy is the released `lstm_bt` checkpoint: one training run at seed 1234 for 200 epochs. It uses $R = 8$ rollouts per polygon, Bradley–Terry temperature $\alpha = 0.05$, and reward weights $\lambda = 1.0$ and $\rho = 3.0$ at the gate $\tau = 0.99$ (Eq. (3)); all PO coverage rewards are computed from discretized-visibility sampling ($M = 500$), after which the encoder is frozen. During development we compared several training *recipes*: REINFORCE baselines (Section 4.1) and alternative fine-tuning objectives, where PO/BT gave the best `train`-set performance. Several runs of that recipe were then trained; the released policy is the best of them, and within a run the retained checkpoint is likewise the best, both scored on `train` by greedy-decoded coverage *minus* the selected vertex fraction. That criterion is Eq. (3) without its two gate terms, and the penalty is what makes it usable: coverage alone is maximized by over-guarding (Section 8). Both choices use only `train`, so `test` and the OOD splits play no part in them; `dev` is reserved for the decision threshold and the canonical start rule. Except for the replication policies of Section 6.4, this is the checkpoint behind every result that follows, including the probe.

Inputs: a polygon's vertices are consumed by the LSTM in the source-library file order, with no canonical start vertex and no orientation (clockwise/counterclockwise) imposed, and the coordinates are min–max normalized per polygon to $[0, 1]^2$ before the encoder. The normalization makes the policy invariant to translation and to axis-aligned rescaling by construction; it is *not* invariant to rotation, to reflection, or to a cyclic re-indexing of the vertices, and we find empirically that these symmetries are not acquired during training either. Of the three, only the vertex-order dependence admits a cost-free fix, by canonicalizing the traversal at inference; we quantify all of this in Section 8.

Targets: for each training polygon we obtain $S^{\star}_{\mathrm{LS}}(P)$ by running local search on the policy's seed under a coverage gate $\tau = 0.99$ against a discretized-visibility reference (Section 4.1); exact coverage-area computation is reserved for evaluation. These targets are *not* produced by a policy-independent classical solver: they come from a best-improvement editor (add/remove/swap) initialized at the policy's own greedy-decoded seed and scored on discretized visibility. We refer to this policy-seeded refinement as "LS" throughout; it is the same edit family whose *learned* counterpart we analyze in Section 6.5, here run to its best-improvement optimum as an oracle teacher rather than learned.

Sizes: the model dimensions of Section 4.2 take the following values:

working width d	128
self-attention blocks L	3
attention heads H	8
FFN expansion factor f	2
policy parameters	$\approx$ 364k
SetPredictor parameters	464,001

Optimization: weighted BCE (Eq. (10)) with positive-class weight w_+ set to the empirical non-guard-to-guard ratio on `train` (5.77 for the released policy's targets, and 5.77–5.90 for the independently trained policies of Section 6.4, each of which generates its own targets), 60 epochs, batch size 32, learning rate 3×10^{-4}, AdamW [35]. Every SetPredictor variant, the headline probe, the no-encoder ablation, and the capacity-ladder rungs (Section 6.2) is four independent runs with probe seeds $\{1234, 11, 22, 33\}$ (reported as mean $\pm$ std); all figures display seed 1234. These four seeds are always SetPredictor *probe* seeds; the underlying PO/BT policy is the single run described above, not re-seeded. We do *not* select probe checkpoints: every reported probe is the final epoch of its run, a fixed rule set before training. The decision threshold is the one hyperparameter `dev` does select, and `test`, `ood`, and `ood-large` are touched only after training and threshold selection are complete. The headline threshold $t = 0.20$ is the coverage-favoring end of the swept range $\{0.20, 0.25, 0.30\}$, selected on `dev` before any `test` or OOD use. The 0.95 coverage gate does not by itself single it out, since every threshold in the sweep clears that gate on all `dev` polygons; what selects $t = 0.20$ is the stricter criterion we report alongside it, namely the count remaining below the 0.99 gate and the worst-case coverage $\min_P \mathrm{Cov}_P(S)$, on both of which it is the best of the three at the cost of the highest guard count. We report all three thresholds rather than tune t per split, and the wider curve in Section 6.2 shows what the choice costs.

5.3. Evaluation protocol and metrics

For each polygon we greedy-decode the policy's EOS-truncated seed and evaluate it with CGAL exact polygon visibility [34]; separately, we run one SetPredictor pass, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with CGAL. Let N be the partition size. We report mean coverage $\frac{1}{N}\sum \mathrm{Cov}_P(S)$, the normalized guard count $|S|/n$, and the approximation ratio $|S|/\mathrm{OPT}$ against

the vertex-guard optimum. Because the mean hides the tail, we also report the per-polygon counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$ and the worst case $\min_P \mathrm{Cov}_P(S)$. We distinguish two notions throughout. *Exact feasibility* is full coverage, $\mathrm{Cov}_P(S) = 1$, as the problem defines it (Eq. (2)); we reserve the words *feasible* and *feasibility* for this exact sense. In the reported counts this is tested as $\mathrm{Cov}_P(S) \geq 1 - 5 \times 10^{-5}$, absorbing floating-point error in the CGAL area ratio. Separately, the counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ report a family of *coverage gates*; we single out $c = 0.95$ as the 0.95 *coverage gate*, a near-feasibility reporting threshold, and report the gate-clearing rate $\#\{\mathrm{Cov}_P(S) \geq 0.95\}/N$ with Wilson 95% confidence intervals. Clearing the 0.95 gate does *not* mean a polygon is feasible in the exact sense. These reporting gates are distinct from the training-time coverage gate $\tau = 0.99$ of Eq. (3), which shapes the reward rather than scoring the output. Where we compare the policy's and the probe's gate-clearing rates on the same polygons, we test the paired per-polygon outcomes with an exact McNemar test. As noted in Section 5.1, OPT values are taken from the vertex-guard optimal solutions distributed with the AGPVG library (for library instances) or computed via ILP (for the `random` class). For 79 of the `ood-large` polygons (all with $n \geq 800$) no optimum is available, most are library instances whose distributed solutions do not cover them (they are open at the source), and for the remainder our ILP did not terminate within budget. Restricting $|S|/\mathrm{OPT}$ to the 206 solved instances would bias the ratio toward the easier 600–700-vertex polygons, so the headline `ood-large` results (Table 9) report coverage and the normalized guard count $|S|/n$ over all 285 polygons and no approximation ratio. Table 10 does carry an $|S|/\mathrm{OPT}$ column there, averaged over the 206 solved instances only; it is read within a row, never across splits.

Computational cost. All experiments were run on a single workstation with an AMD Ryzen 9 3900X CPU, 64 GB RAM, and an NVIDIA GeForce RTX 2080 SUPER GPU (8 GB VRAM). Training the SetPredictor probe is light: about 5 s per epoch, $\approx$ 5 minutes for a 60-epoch run on the GPU above (the four-seed ensemble, $\approx$ 20 minutes); the frozen PO/BT policy is reused rather than retrained. The geometric preprocessing (the exact per-vertex CGAL visibility polygons and the $M = 500$ discretized-visibility matrix) is computed once per polygon and cached for the $\sim$9.9k library polygons, so it is not repeated across rollouts, edits, or seeds. At inference the method is a single geo-free forward pass, tens of milliseconds on the GPU above; Section 6.6 times it in full and sets it against the classical visibility-based pipeline. Exact CGAL coverage is used solely for evaluation and is the dominant offline cost, scaling with the guard-set size.

Method	Mean cov	$\#\{\text{Cov} \geq 0.95\}/N$	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert/\text{OPT}$
Classical baseline (non-learned)				
Greedy	0.9994	362/362	0.1524	1.0092
Learned policy / probe				
Pretrained pointer (seed)	0.9689	293/362 (0.766,0.847)	0.1664	1.0885
SetPredictor full ($t = 0.20$)	$\mathbf{0.9912} \pm 0.0024$	$349.8 \pm 3.5/362$	0.2475 ± 0.0050	1.6208 ± 0.0299
SetPredictor, no-encoder ($t = 0.20$)	0.9976 ± 0.0004	$361.5 \pm 0.5/362$	0.4517 ± 0.0432	2.9405 ± 0.2654
Oracle editor (SetPredictor target, policy-seeded)				
LS on policy seed	0.9948	362/362	0.1369	0.8854[2]

Table 2: Held-out evaluation on `test` (362 polygons). Classical greedy is the only non-learned anchor. *seed* is the PO/BT policy decoded greedily; *LS on policy seed* is its local-search refinement, the SetPredictor's supervised target, not a classical baseline (Section 5.2). SetPredictor *full* and *no-encoder* are the headline probe and its zeroed-embedding ablation, mean ± std over four probe seeds at $t = 0.20$. At this common threshold the two probes sit at different operating points, so the gate counts are not an encoder comparison; Section 6.2 makes that at matched guard budget. Wilson 95% CI shown only where the proportion is informative.

6. Results

6.1. The policy places guards

The geo-free policy on its own (Table 2, *seed* row) places guards at near-classical cardinality but clears the 0.95 coverage gate on most, not all, of the `test` polygons (Wilson 95% CI in the table). The LS oracle (bottom row) clears it everywhere at lower cardinality still, yet is not geo-free: it reads discretized visibility at every add/remove/swap step and serves here only as the probe's training target, not as a competitor. Vertex-guard placement is therefore learnable, to a degree, by a method that never reads an explicit visibility object at inference. The polygons left below the gate are this paper's central motivating observation: their distribution is shown in Section 6.2 and the method gives no coverage guarantee on its own. Figure 2 shows the seed, the seed plus probe at $t = 0.20$, and the optimum side by side for an in-distribution and an out-of-distribution polygon.

6.2. Reading the encoder shortens the tail

The policy's mean coverage hides a bimodal distribution: it reaches full coverage on only a handful of polygons and trails a long left tail, while the SetPredictor at $t = 0.20$ shortens that tail sharply and lifts the bulk above 0.99 (Table 3). The improvement is a genuine shift rather than a trade, which the table's marginal counts cannot show: on the displayed seed the probe lifts 57 previously-failing polygons back over 0.95 while pushing 5 the other way (paired exact McNemar test on the 62 discordant polygons, $p \approx 3 \times 10^{-12}$); the same test is applied at OOD scale in Section 6.3. The shift also holds under the strict full-coverage requirement ($\text{Cov}_P(S) = 1$), so the gain is not an artifact of

[2]The oracle editor halts at the disc-vis gate ($\tau = 0.99$) and slightly undercovers, so it spends fewer guards than the *full-coverage* optimum, an artifact of the training reward, not a loose OPT.

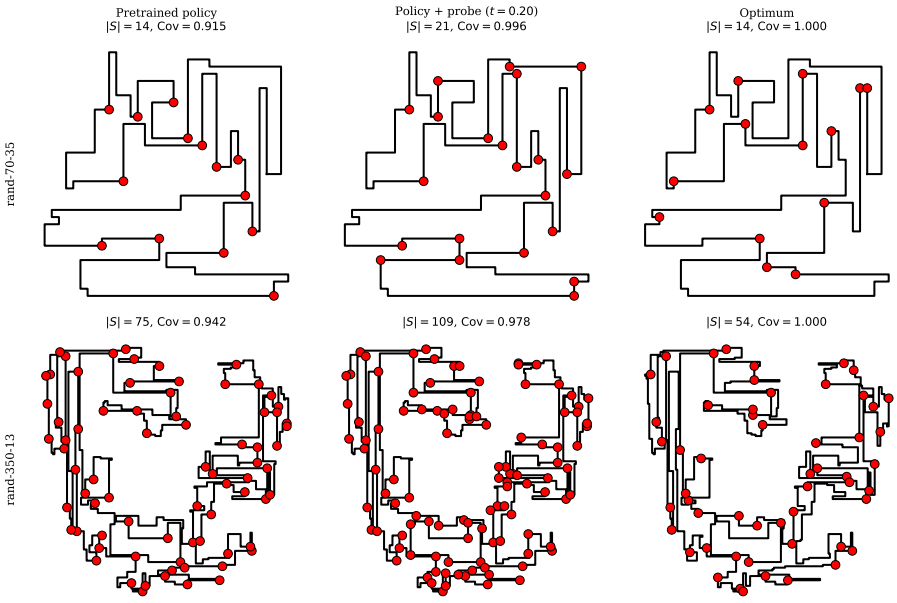

Figure 2: Two polygons drawn with the policy's seed (left), the policy plus the SetPredictor probe at $t = 0.20$ (middle), and the optimum (right); guard vertices are marked, with $|S|$ and coverage printed per panel. Both are cases where the seed falls below the 0.95 gate and the probe lifts it above. Neither is a best case: both are typical examples, picked by size from the polygons the probe rescues. Row 1 is in-distribution ($n = 70$), row 2 out-of-distribution ($n = 350$, about 1.8× the largest training polygon).

Method	#{Cov = 1}	#{Cov ≥ 0.999}	#{Cov ≥ 0.99}	#{Cov ≥ 0.95}	min Cov
Pretrained pointer (seed)	3	15	97	293/362	0.830
SetPredictor $t = 0.20$ (full)	61	128	302	345/362	0.857
SetPredictor $t = 0.20$, no-encoder	151	206	330	362/362	0.951

Table 3: Per-polygon coverage distribution on `test` (362 polygons). Counts are polygons reaching each coverage threshold; *min* $\mathrm{Cov}_P(S)$ is the worst polygon. The no-encoder ablation reaches the stricter gates more often than the full probe here only because at this common threshold it selects 1.8× as many guards; the encoder's contribution appears once the two are placed at matched guard budget (Section 6.2). All rows are the displayed seed (1234) at the headline threshold; Table 2 gives the four-seed mean±std and Figure 3 the threshold sweep.

reading only the loose 0.95 gate, though exact coverage remains the minority case, the limitation Section 8 returns to.

The next question is whether closing that tail is the encoder's doing, or whether coordinates alone would suffice. We isolate the encoder by retraining the probe with the frozen embedding h_i masked to zero, holding architecture, parameter count, optimizer and schedule identical (Section 5.2). The two cannot then be compared at a common threshold. Masking the embedding also changes how many vertices the probe selects: at $t = 0.20$ the ablation returns nearly twice the full probe's guard fraction, and a larger set raises coverage on its own,

so the failure counts track the guard budget rather than the representation (Table 4). Nor is the concern hypothetical: the conditions' cost curves *cross*, so which one looks better is decided by where the threshold is put (Table 5a). We therefore report the fixed-t form for completeness and draw the encoder's contribution at matched guard budget instead.

The encoder's contribution is instead read at *matched guard budget*. We sweep all four conditions over $t \in [0.05, 0.80]$ (Table 5a) and then read each at a common guard cost (Table 5b). At every one of those budgets the full probe leaves a shorter tail than no-encoder, by a margin five to nine times the seed-to-seed spread. One row of the sweep needs no matching at all: already at $t = 0.30$ the full probe uses fewer guards than no-encoder *and* leaves about two-fifths fewer polygons below the 0.99 gate (Table 5a). Read as cost, reaching the same tail costs no-encoder about $1.45\times$ the full probe's guards, coords-only about $1.75\times$, and no-seed about $1.1\times$. It is the frozen encoder, not the policy's seed and not the coordinates, that lets the probe turn guards into coverage economically. Those multiples rank the four conditions (full cheapest, then no-seed, no-encoder and coords-only) and that ranking holds for every probe seed at every budget.

Repeating the sweep on the three independently trained policies of Section 6.4 reproduces this. Wherever the full probe and the no-encoder ablation can be compared at the same guard budget, the full probe leaves the shorter tail; at the widest gap the ablation leaves several times as many polygons below the gate. Stated as cost, removing the encoder demands more guards for the same tail on all four policies, from about +30% to +81% depending on the policy: the penalty is universal but its size is not. C1 therefore rests on four training runs rather than one.

Two caveats. The claim is about *matched cost*, not attainable coverage: given enough guards the no-encoder ablation does reach essentially complete coverage, but pays more than twice the full probe's guard cost to get there. And at a fixed threshold the comparison runs the other way: at $t = 0.20$ the ablation clears the 0.95 gate on every polygon while the full probe leaves seventeen below it (Table 3), because there it is spending nearly twice the guards to do so.

Two conditions deserve comment. The no-encoder ablation keeps the policy's seed indicator, so its recovered coverage could in principle be credited to the decoder's guard set; completing the 2×2 by masking the seed as well settles that, since doing so barely moves the probe while masking the encoder is what costs guards. The coords-only condition is the floor: the same probe, reading coordinates alone. It never becomes economical, held to the full probe's budget it fails on more than three times as many polygons, and its apparently short tail at $t = 0.20$ is bought by selecting nearly three-quarters of all vertices.

Probe capacity. Since the SetPredictor is itself a small Transformer, one might worry that *it* computes the geometry while the encoder merely passes coordinates through, the standard caveat in the probing literature [10, 12]. A capacity ladder rules this out (Table 6): a linear readout, an attention-free MLP, a single self-attention layer, and the full three-layer probe, each reading

Probe inputs	#{Cov < 0.99}	min Cov	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT
SetPredictor full	57.8 ± 4.8	0.551 ± 0.327	0.25 ± 0.01	1.62 ± 0.03
no-seed (encoder + coords)	45.5 ± 13.4	0.547 ± 0.335	0.30 ± 0.02	1.94 ± 0.12
no-encoder (seed + coords)	22.0 ± 6.0	0.941 ± 0.023	0.45 ± 0.04	2.94 ± 0.27
coords-only	0.8 ± 0.8	0.962 ± 0.044	0.72 ± 0.01	4.64 ± 0.06

Table 4: Encoder–seed ablation on `test` (362 polygons), all probes at $t = 0.20$, four-seed mean ± std. The rows are the 2×2 on/off combinations of the two *ablated* inputs of Eq. (7), the frozen encoder embedding and the policy's seed indicator; the coordinates are read in every row. An absent input is masked to zeros, leaving architecture and parameter count unchanged. Read the tail and cost columns together: at a fixed threshold the rows without the encoder show a short tail only by selecting far more vertices, so their tail reflects set size rather than selection quality. The encoder comparison is drawn at matched guard budget instead (Table 5), where the ordering reverses.

	full		no-seed		no-encoder		coords-only	
	$\lvert S\rvert$/OPT	#fail	$\lvert S\rvert$/OPT	#fail	$\lvert S\rvert$/OPT	#fail	$\lvert S\rvert$/OPT	#fail
(a) swept operating curves; rows are the threshold t								
0.05	2.01	28 ± 7	2.63	20 ± 8	5.17	0 ± 0	5.45	0 ± 0
0.10	1.81	40 ± 4	2.29	28 ± 10	4.40	2 ± 1	5.04	0 ± 0
0.15	1.70	50 ± 6	2.09	37 ± 13	3.65	6 ± 1	4.81	1 ± 1
0.20	1.62	58 ± 5	1.94	46 ± 13	2.94	22 ± 6	4.64	1 ± 1
0.25	1.56	65 ± 4	1.84	54 ± 14	2.27	63 ± 16	4.48	2 ± 1
0.30	1.50	74 ± 4	1.74	64 ± 13	1.77	126 ± 22	4.31	4 ± 3
0.35	1.45	86 ± 2	1.66	75 ± 14	1.51	166 ± 23	4.06	8 ± 2
0.40	1.41	96 ± 4	1.58	87 ± 15	1.37	200 ± 17	3.68	18 ± 6
0.45	1.36	109 ± 6	1.51	100 ± 14	1.27	222 ± 14	3.11	37 ± 10
0.50	1.32	119 ± 7	1.45	114 ± 16	1.21	239 ± 12	2.43	94 ± 19
0.55	1.28	133 ± 9	1.39	130 ± 16	1.17	249 ± 11	1.79	202 ± 31
0.60	1.23	153 ± 10	1.33	150 ± 20	1.13	255 ± 10	1.25	278 ± 27
0.70	1.13	202 ± 13	1.21	197 ± 23	1.09	270 ± 10	0.53	328 ± 8
0.80	0.98	269 ± 18	1.08	254 ± 14	1.06	284 ± 8	0.24	343 ± 3
(b) #fail at matched guard budget; rows are $\lvert S\rvert$/OPT								
1.95		32 ± 5		43 ± 1		101 ± 14		175 ± 12
1.70		50 ± 6		69 ± 4		136 ± 14		215 ± 9
1.50		76 ± 7		101 ± 5		170 ± 10		245 ± 8
1.30		126 ± 12		159 ± 6		216 ± 3		270 ± 9

Table 5: Encoder–seed ablation at *matched guard budget* on `test` (362 polygons); Table 4 shows the same four conditions at a fixed threshold. Entries are means over four probe seeds. #fail is the number of polygons below the 0.99 gate under exact CGAL; its ± std is shown, that of $\lvert S\rvert$/OPT omitted for width (at most 0.28, usually under 0.10). *(a) Swept curves:* a common t buys each condition a different number of guards, so rows cannot be read across conditions, hence panel (b). *(b) Matched budget:* each seed is interpolated to a common $\lvert S\rvert$/OPT along its own curve and the four are then averaged. Because (a) instead averages at a fixed t, where the seeds sit at different costs, the same condition can show a different count at what looks like the same budget; (b) is the form the matched-budget claim rests on. Values in (b) are read between swept thresholds, so the narrow gaps between adjacent conditions at the extreme budgets depend on that reading in a way the full-versus-no-encoder gap does not.

the same frozen features and each trained on `train` and scored on `test`. We score them by ROC-AUC, which asks only whether guard vertices rank above non-guards and so needs no threshold; at any fixed t the four capacities would sit at different guard budgets, the same confound as above. Even the zero-

hidden-layer linear rule reaches ROC-AUC ≈ 0.84, and it cannot be credited with computing geometry itself [9], so the guard-relevant structure is already linearly accessible in the frozen embedding. Ranking quality then rises with capacity but saturates early: the attention-free MLP recovers about 84% of the linear-to-full gain, and the two attention rungs add only the remainder. PR-AUC, which unlike ROC-AUC is sensitive to the 0.16 positive-class prior and so cannot be flattered by the abundant negatives, gives the same reading: there the MLP recovers about 80% of the span. The capacity conclusion therefore does not rest on the choice of metric. A modest nonlinear readout is thus enough and the full Transformer's capacity adds little, so the probe reads the structure rather than building it.

Probe capacity	Params	ROC-AUC	PR-AUC
Linear (logistic)	129	0.837 ± 0.000	0.531 ± 0.000
Attention-free MLP	66,561	0.909 ± 0.001	0.633 ± 0.003
1 self-attention layer	199,041	0.912 ± 0.003	0.643 ± 0.011
Full SetPredictor (3 layers)	464,001	0.923 ± 0.002	0.658 ± 0.015

Table 6: Probe-capacity ladder: every rung reads the same frozen features and is trained on `train` against the LS target, then scored per-vertex on all 362 `test` polygons; the linear rung is fit on a 600-polygon subset of `train`, ample for its 129 parameters. The $\pm$ is spread across four probe seeds; the linear rung is a deterministic logistic fit, so it has none. Most of the linear-to-full gain arrives with no self-attention at all.

Linear readout over	dim	ROC-AUC	PR-AUC
Coordinates (x_i, y_i)	2	0.537 ± 0.013	0.178 ± 0.010
Analytic geometry	10	0.753 ± 0.014	0.345 ± 0.013
Frozen encoder h_i	128	0.843 ± 0.009	0.581 ± 0.027
Encoder + geometry	138	$\mathbf{0.872} \pm 0.008$	0.620 ± 0.021

Table 7: Feature-set control for the capacity ladder: one linear rule (L2 logistic) read over four geo-free feature sets, so the conditions differ only in their inputs. Unlike Table 6, whose rungs are trained on `train`, these rules are fit *within* 200 `test` polygons under 5-fold polygon-grouped cross-validation, so no vertex is scored by a rule that saw its own polygon; the $\pm$ is across those folds. That difference in protocol, not in the features, is why the encoder row and Table 6's linear rung differ slightly. *Analytic geometry* is ten $O(n)$ coordinate functions (angles, a reflex flag, edge lengths, corner span, centroid distance, n), none consulting a visibility oracle.

A feature-set control. The ladder holds the features fixed and varies the probe; the complementary control varies the features. Interior angles and reflexness are the classical signals for guard placement and are computable from the coordinates in $O(n)$ with no visibility oracle, so a readout over them is geo-free by the definition of Section 2.3. Table 7 reads four feature sets with one linear rule, cross-validated within 200 `test` polygons rather than trained on `train` as the ladder is. Ten analytic features land far above raw coordinates, recovering about 71% of the encoder's gain over them, so the encoder's margin over hand-designed geometry is real but narrower than a comparison against coordinates alone would suggest. The reverse direction is the more informative one: append-

ing those same ten features to the frozen embedding improves the readout in every fold, by more than twice what the two attention rungs of Table 6 buy over an attention-free readout. The encoder therefore neither reduces to standard geometric descriptors nor subsumes them; what it carries is partly complementary to them, not reflexness rediscovered. Appending the interior angle by itself recovers about half of that gain, in every fold, so the deficit is not spread thinly over the ten features: the encoder is missing a rigid-motion invariant this elementary, which is consistent with its degradation under rotation (Section 8). Being linear readouts, these conditions compare linearly decodable information rather than information in principle.

The cost of closing the tail. The threshold t is the knob, and the shape of that trade-off matters for how the method should be read (Figure 3). Guard cost falls steeply as t rises while coverage decays slowly, so the headline $t = 0.20$ is the expensive end rather than the only usable point: by $t = 0.50$ the probe still clears the 0.95 gate on 337 of 362 polygons against the policy's 293, at appreciably lower guard cost (Table 5a). The gain is bounded, though, and the curve says where it stops: by $t = 0.70$ the probe has converged to the policy seed on both axes, and by $t = 0.80$ it is strictly worse, cheaper but covering less on every measure. Figure 4 gives the same picture in and out of distribution. We report the trade-off rather than fix one operating point.

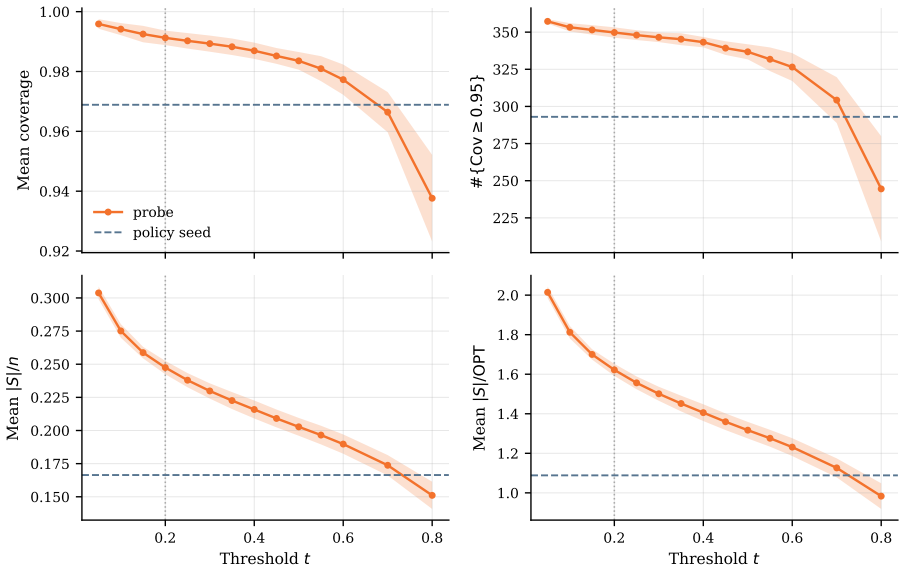

Figure 3: The probe's coverage–cost operating curve over the threshold t, single pass, on `test` (362 polygons). Top row outcome, bottom row cost; lines are the four-seed mean with ±1 std bands, the dashed line the t-independent policy seed, the dotted vertical the headline $t = 0.20$. The two cost panels are one curve at two scales ($|S|/n$ and $|S|/\mathrm{OPT}$ agree to within 2.3%). The bands widen with t: the cheap end is the less stable one.

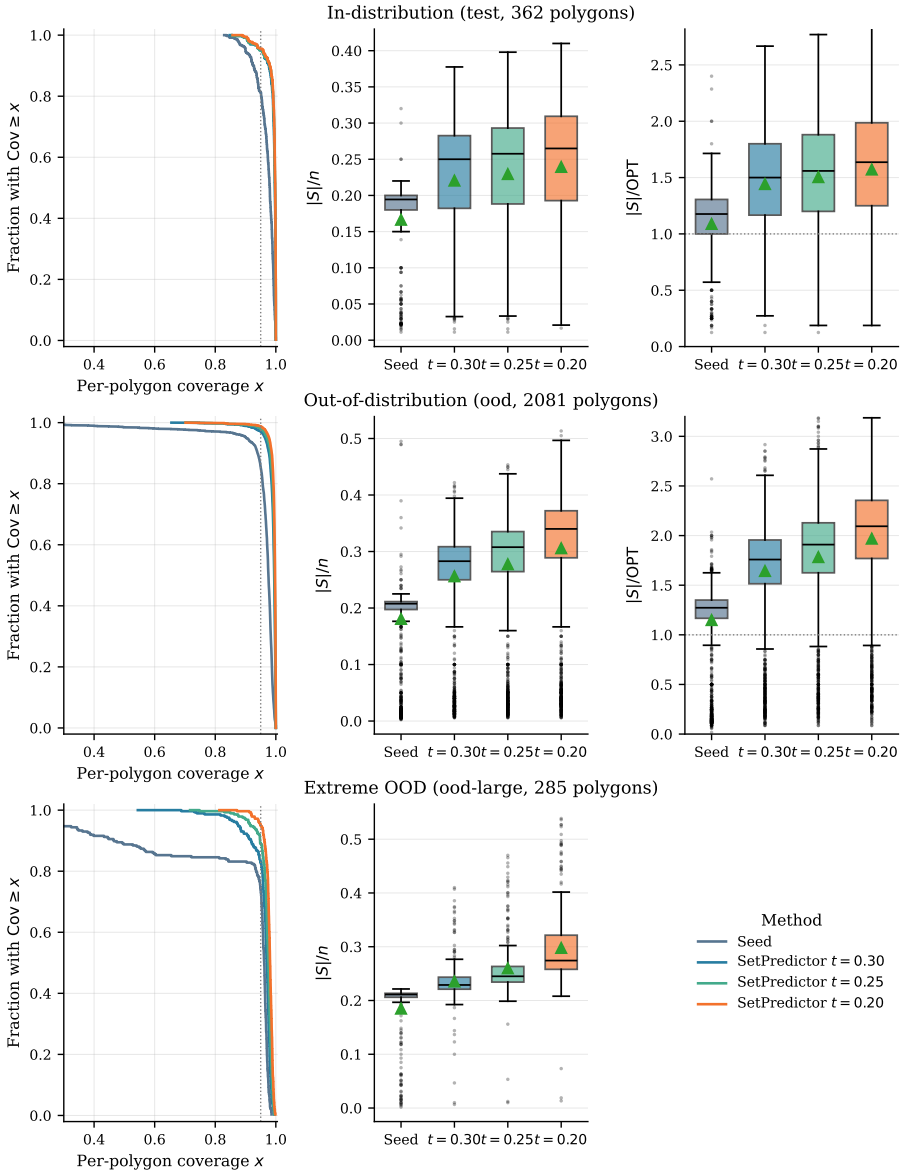

Figure 4: Distributional view across `test` (top, 362 polygons), `ood` (middle, 2081) and `ood-large` (bottom, 285). *Left:* per-polygon coverage as a complementary CDF, dashed line at the 0.95 gate. *Centre:* $|S|/n$ for the policy seed and three probe thresholds. *Right:* $|S|/\mathrm{OPT}$, omitted for `ood-large` where OPT is unavailable for the larger polygons. Displayed seed 1234.

Method	Mean cov	#{Cov ≥ 0.95}/N	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
Pretrained pointer (seed)	0.9567	1778/2081 (0.839,0.869)	0.1807	1.1488
SetPredictor full ($t = 0.20$)	**0.9936** ± 0.0009	2058.2 ± 7.3/2081	0.3023 ± 0.0055	1.9486 ± 0.0363
SetPredictor, no-encoder ($t = 0.20$)	0.9874 ± 0.0023	2047.8 ± 4.7/2081	0.3670 ± 0.0335	2.3571 ± 0.2149

Table 8: Out-of-distribution evaluation on `ood` (2081 polygons, n up to 1000). Rows as in Table 2: the policy seed, the full probe, and the no-encoder ablation, both probes the four-seed mean ± std at $t = 0.20$.

6.3. Out-of-distribution generalization

The next test of the representation is the `ood` split, a larger held-out set that spans the training size range and extends well beyond it, reaching $n = 1000$, about five times the largest training polygon. The policy alone drops in mean coverage, clearing the 0.95 gate on 85% of the split; the probe at $t = 0.20$ lifts that to 99%, roughly an order-of-magnitude reduction in failures (Table 8). The improvement is significant for every probe seed individually (exact McNemar test on the paired per-polygon outcomes, $p < 10^{-74}$ in all four cases), and the per-seed Wilson 95% intervals for the gate-clearing rate all lie above 0.977. Nor is it an artifact of the in-distribution-size majority of the split: restricted to the genuinely larger-than-training polygons ($n > 198$), both rates are essentially unchanged. This is the evidence behind C2, and Section 6.4 shows it is not specific to the policy we released. What is *not* stable is the residual failure count, which varies by a factor of nearly three across probe seeds (Table 8).

The encoder comparison needs more care here. On this policy the encoder gains on *both* axes at a common threshold, which it does not do in distribution: at $t = 0.20$ the full probe uses fewer guards than the no-encoder ablation *and* leaves fewer polygons below the gate (Table 8). That double gain is specific to this policy, however. Repeating the same fixed-threshold comparison on three independently trained policies reproduces it on one and reverses the tail on the other two, where the ablation again clears more polygons by spending more guards (Section 6.4). The guard-budget confound therefore does not reliably lift out of distribution; it lifted on the policy we happened to release. What generalizes is the matched-budget reading, which holds on all four policies in distribution (Section 6.2). Out of distribution we swept the full threshold range for the released policy only, and there the matched-budget margin is wider than the fixed-threshold one. Figure 4 (middle and bottom rows) shows the matching coverage and cost distributions.

Method	Mean Cov	min Cov	#{Cov ≥ 0.95}/N	Mean $\lvert S\rvert/n$
Pretrained pointer (seed)	0.8678	0.038	210/285	0.1849
SetPredictor full ($t = 0.20$)	**0.9734** ± 0.0064	0.766 ± 0.106	265.8 ± 10.4/285	0.2890 ± 0.0178
SetPredictor, no-encoder ($t = 0.20$)	0.9379 ± 0.0085	0.105 ± 0.079	255.8 ± 5.5/285	0.3069 ± 0.0223

Table 9: Extreme out-of-distribution evaluation on `ood-large` (285 polygons, n from 600 to 2250), coverage scored by exact CGAL at $t = 0.20$ over all 285. The approximation ratio is omitted because OPT is unavailable for 79 polygons ($n \geq 800$; Section 5.3). The full-probe and no-encoder rows are four-seed mean ± std; their min Cov is the mean ± std of the four per-seed worst-polygon coverages.

Extreme out-of-distribution. On the `ood-large` split (n from 600 to 2250, up to about eleven times the largest training polygon), the worst-case coverage is the more informative metric, as the mean can look reasonable while the tail collapses (Table 9). The pretrained seed leaves its worst polygon almost entirely uncovered, and the no-encoder ablation barely moves it: its worst-polygon coverage stays collapsed on every probe seed. The full probe lifts that worst polygon on every seed, to roughly seven times the ablation's worst-case coverage on the four-seed mean, though still well short of the gate (Table 9). This is where the encoder's contribution is clearest at this scale, and it is the only axis on which it is clear. Neither condition clears the gate everywhere: the full probe is ahead on three of the four probe seeds, but the margin sits inside its own seed spread, so we do not read a gate-count advantage from this split. Nor does the probe buy a guard saving here, unlike on the other two splits, the two conditions' approximation ratios differ by less than the seed spread. Eleven times beyond the training range, then, the encoder's advantage is confined to the worst case.

6.4. Replication across independently trained policies

Everything above reads one policy. Because the paper's question is what *a* reinforcement-trained encoder holds, a single training run cannot separate the claim from the run, so we trained three further PO/BT policies from scratch under the identical recipe (seeds 11, 22, 33; Section 5.2) and, for each, trained its own full and no-encoder probes on its own local-search targets. A probe is only meaningful relative to the encoder it reads, so the policies cannot share probes. Table 10 reports the three replication policies on all three splits; the released policy is the one already tabulated in Tables 2, 8 and 9, and is not repeated there.

Two things replicate and one does not. The three differ appreciably in quality from each other and from the released policy, both in gate-clearing and in approximation ratio (Table 10 against Table 2), so this is a genuine sample of the recipe rather than a re-run of one outcome. First, the probe closes the coverage tail on *every* policy, and by the largest margins out of distribution, where each policy's below-gate count falls by roughly an order of magnitude or more. This is the evidence behind C2, and it now rests on four training runs. Second, the matched-budget encoder comparison of Section 6.2 survives the change of policy. What does *not* replicate is the fixed-threshold encoder comparison out of distribution, discussed above.

One row needs reading with care. On `ood-large` policy 22 does not decode a guard set so much as flood one: its seed already selects nearly three-quarters of every polygon's vertices and trivially clears the gate on all of them (Table 10). It has no tail to close, and the probe, which selects *fewer* guards, necessarily gives coverage back. We exclude that cell from the encoder comparison rather than scoring it as a failure of the probe, on the same reasoning that makes a fixed threshold the wrong place to compare conditions: a method that already spends most of the budget cannot be beaten on coverage, and beating it there would mean nothing. Among the three policies that do leave a tail on this split, the full probe leaves the shorter one in all three.

	policy seed				+ full probe				+ no-encoder			
Policy	cov	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT	gate	cov	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT	gate	cov	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT	gate
test (362 polygons)												
11	0.968	0.160	1.04	290/362	0.989	0.222	1.45	342/362	0.998	0.419	2.75	362/362
22	0.967	0.171	1.12	286/362	0.992	0.260	1.70	353/362	0.994	0.380	2.47	361/362
33	0.962	0.169	1.09	264/362	0.996	0.308	2.03	359/362	0.996	0.498	3.25	361/362
ood (2081 polygons)												
11	0.966	0.173	1.11	1688/2081	0.992	0.299	1.92	2051/2081	0.992	0.371	2.41	2075/2081
22	0.977	0.298	2.37	1861/2081	0.995	0.372	2.69	2053/2081	0.993	0.424	3.12	2074/2081
33	0.964	0.222	1.42	1682/2081	0.998	0.377	2.46	2078/2081	0.989	0.402	2.58	2050/2081
ood-large† (285 polygons)												
11	0.947	0.200	1.27	200/285	0.979	0.283	1.87	278/285	0.973	0.316	2.10	271/285
22	1.000	0.719	4.67	285/285	0.962	0.586	3.86	222/285	0.991	0.697	4.51	262/285
33	0.875	0.569	3.68	220/285	0.984	0.585	3.77	266/285	0.864	0.529	3.54	119/285

Table 10: Replication across the three PO/BT policies trained from scratch under the same recipe (seeds 11, 22, 33), $t = 0.20$, coverage by exact CGAL. Each policy has its own full and no-encoder probe trained on its own targets; the released policy (1234) is tabulated in Tables 2, 8 and 9 and is not repeated here. "gate" is $\#\{\mathrm{Cov}_P(S) \geq 0.95\}/N$. †On `ood-large` the $|S|$/OPT column averages only the 206 of 285 polygons that have a vertex-guard optimum, so it is not comparable across splits; the other columns cover all 285. At this fixed threshold the coverage and cost columns are interdependent; the encoder conclusion is drawn at matched budget (Table 5) rather than from this table.

6.5. Alternatives to the probe: decode search, editors, and the fixed point

Decode-time search does not close the tail. The simplest alternative to the probe is to decode the policy better rather than read its encoder. We test this directly (Table 11): for each `test` polygon we draw $K = 32$ stochastic rollouts from the frozen policy and select among them geo-free, by length-normalized log-likelihood, with no visibility oracle. The result differs from greedy by a single polygon (Table 11), because the policy's maximum-likelihood decode already *is* the greedy seed: likelihood-ranked search never prefers a more-covering set, and occasionally prefers a less-covering one, lowering the worst polygon. Letting an *oracle* pick the highest-coverage sample of that same pool (classical active search, no longer geo-free) raises gate-clearing but still leaves a substantial tail below 0.99 (Table 11), far short of what reading the encoder achieves (Table 3). The tail is thus not a decoding artifact that more search removes: closing it requires reading the encoder, and the probe pays for that in guards rather than in search.

Decode of the frozen policy	$\#\{\mathrm{Cov} \geq 0.95\}/N$	$\#\{\mathrm{Cov} < 0.99\}$	Mean Cov	min Cov	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT
greedy (seed)	293/362	265	0.969	0.830	0.166	1.09
best-of-32, likelihood (geo-free)	294/362	266	0.967	0.557	0.166	1.09
best-of-32, coverage oracle (*not* geo-free)	333/362	199	0.981	0.845	0.171	1.12

Table 11: Decode-time search on `test` (362 polygons), all reading the same frozen PO/BT policy. *greedy* is the seed of Table 2. *best-of-*32*, likelihood* draws $K = 32$ stochastic rollouts and keeps the one with the highest length-normalized log-likelihood, a geo-free decode search with no visibility oracle. *best-of-*32*, coverage oracle* instead keeps the highest-coverage sample (tie-break: fewer guards); it consults exact coverage to choose and is therefore *not* geo-free, reported only as an upper bound on what the policy's own samples contain. Greedy is included in each candidate pool, so neither selection can underperform it on its own ranking metric. Geo-free search matches greedy and does not close the tail; even oracle-assisted search falls well short of the SetPredictor (Table 3).

Before settling on the single-shot probe we tried to improve the seed with a learned *iterative* editor applying ADD/REMOVE/STOP edits. Three architectures of increasing capacity, including one with topology features and an auxiliary visibility-prediction loss, and one with DAgger refresh [32], failed to improve over the seed in any coverage-preserving regime. The failure takes the form of a trade the editor cannot escape. Its best configuration cuts 16% of the guards but loses coverage doing so ($0.969 \to 0.963$ mean), and recovers the whole of local search's gain over the seed, scored on the reward that trades coverage against guard count, for just 0.33 of the polygons where local search improves the seed at all, a *recovery rate* well below replacement quality. Configurations tuned to cut harder do cut harder, up to 22%, but pay for it in coverage, dropping the mean to 0.933 and scoring below the policy's own seed on most polygons (median recovery -0.38). Neither end of that trade is usable, which is what we mean by no coverage-preserving regime: on the held-out split no configuration we trained cut guards without giving up coverage. Three structural reasons recur: errors accumulate because the editor sees clean trajectories in training but its own predictions at inference; a single $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \ldots, n\}$ head couples selection with termination, miscalibrating STOP; and the train/inference state distributions diverge in ways DAgger refresh did not close.

The same teacher-forcing cascade also defeats a *supervised* pointer network, which we trained directly on AGPVG solutions before adopting the reinforcement recipe. Cloning a guard sequence is ill-defined for a second reason beyond the cascade: a polygon has many optimal and near-optimal guard sets (the constraint $\text{Cov}_P(S) = 1$ in Eq. (2) fixes the cardinality, not the membership), so any single target sequence is an arbitrary choice among equivalent solutions, and an autoregressive loss penalizes the model for proposing a different but equally valid one. Forced onto sequences it would not itself generate, the recurrent decoder then drifts into hidden states unseen during training and collapses (mean coverage ≈ 0.50 at roughly $3\times$ the optimum). Preference optimization sidesteps both problems: it ranks the policy's *own* rollouts by reward rather than matching a designated target, so non-unique optima are not in conflict. The single-shot probe likewise has none of these failure modes: no rollout, no stop token, and a per-vertex membership label rather than an ordered sequence, trained and used in the same regime.

More concretely, the probe's supervised targets $S^\star_{\text{LS}}$ are themselves the output of this same edit family run to its best-improvement optimum: an *oracle* editor seeded by the policy. So the contrast is narrower than "editors fail, the probe works". The probe does not reproduce the oracle's guard sets: at the headline threshold it selects roughly $1.8\times$ the cardinality of the very sets it was trained to predict (Table 2), and raising its threshold to match that cardinality costs coverage rather than recovering the oracle's solution (Figure 3). What it reproduces is the oracle's *ranking* of vertices, not its judgement of how many to keep; it reaches comparable coverage by over-selecting. That the probe succeeds at all is therefore not, by itself, evidence about the encoder, since it could merely be distilling the oracle's preferences. What attributes the success to the *representation* is the encoder–seed ablation of Section 6.2, run with the LS tar-

get held fixed. Were the probe merely distilling the oracle, which inputs it reads would not matter, yet it is the encoder, not the decoder's seed, that makes the targets predictable from coordinates alone.

Re-running the probe with its own output as the updated seed indicator leaves the prediction nearly unchanged across the swept threshold range, and the residual drift is systematic rather than noise, but it is not one-directional (Figure 5). It is smallest near $t = 0.5$ and grows on either side of it, changing sign there: at and below the headline thresholds the extra passes end with slightly *more* guards than a single pass, while from $t = 0.6$ upward they end with fewer, and with lower coverage. The guard cost moves monotonically in K at every threshold but $t = 0.5$, where the total movement is within run-to-run noise ($|S|/\mathrm{OPT}$ varies by 0.006); coverage is the noisier of the two axes and at several thresholds dips at $K = 2$ before partly recovering. At no threshold does iterating improve both axes at once, which is what makes a single pass the right default rather than merely a cheap one. What matters for the single-pass design is that coverage barely moves where the probe is operated: at the headline $t = 0.20$ five passes change it by 0.0001, while the guard cost rises by 0.031 in $|S|/\mathrm{OPT}$, about two percent. At $t = 0.8$, where the selected set is smallest and the test most stringent, the guard cost instead falls by 0.133 and coverage by 0.0405. So the fixed-point property holds on the axis that matters at the thresholds we report, and degrades into a slow contraction only as the threshold is pushed toward the decision boundary; it is not exact, and we had earlier expected the drift to shrink monotonically toward the operating point, which the sweep does not bear out. This is the fixed-point property (C4): with the seed indicator already among its inputs, a single pass suffices.

6.6. Runtime and memory: geo-free versus classical

Geo-free inference moves the visibility cost from inference to training; Table 12 shows the effect on real polygons across the size grid used for the learned-pipeline timing. At inference the learned pipeline is one coordinate-only forward pass, a policy encode plus a probe pass, costing at most a few hundred milliseconds even at $n = 2000$ (same workstation as Section 5.3).

Table 12 reports two classical implementations, because the choice that matters for training matters here too. *Exact greedy* is the implementation used for every guard-count and coverage comparison elsewhere in the paper: each candidate's marginal gain is an exact CGAL polygon-set union, with the selection loop accelerated by lazy evaluation of the submodular marginal gains, which leaves the returned guard sets unchanged. *Disc-vis greedy* scores the same additive algorithm on the discretized visibility matrix of Eq. (4) instead, so ranking all remaining candidates is one vectorized array operation per round rather than a CGAL call per candidate. Neither escapes the structure both must build first: before a single guard is chosen, visibility precompute alone already costs one to two orders of magnitude more than the entire learned pass. The two differ in where the rest of the cost sits, not in whether they pay it: exact greedy's selection grows superlinearly and comes to dominate its total, while disc-vis greedy's selection is close to free, leaving visibility construction to dominate. End to

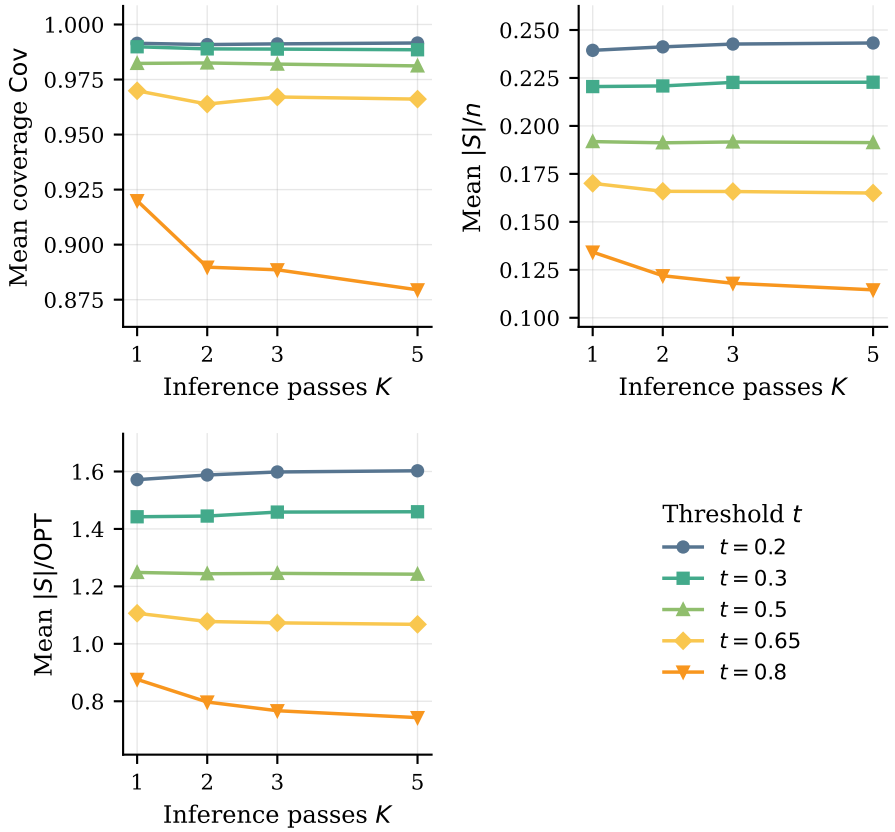

Figure 5: Iterative inference is near-stationary in coverage at the thresholds the probe is operated at, and contracts in guard count only at the highest ones, on the held-out `test` split (362 polygons). Panels show mean coverage (*top left*), $|S|/n$ (*top right*), and $|S|/\mathrm{OPT}$ (*bottom left*) versus inference passes $K \in \{1, 2, 3, 5\}$ (legend, *bottom right*) for a representative ladder drawn from the eight swept thresholds, spanning the headline operating point to the decision boundary. Drift is smallest near $t = 0.5$ and grows on both sides of it, changing sign: extra passes add guards below it and shed them above (bounds in Section 6.5).

end, exact greedy runs two to three orders of magnitude above the learned pass, and disc-vis greedy, the fastest classical alternative we could construct, one to two (Table 12), so the geo-free advantage holds against an optimized classical baseline rather than a naive one.

That speed is not free, and what it costs is not a little accuracy. Disc-vis greedy scores coverage as the fraction of 500 fixed sample points its guards see, so once all of them are covered no vertex has any marginal gain left: the search halts and reports coverage of exactly 1. That is a ceiling rather than a stopping choice, and extra guards cannot lift it, since past that point nothing distinguishes them. Driven to that ceiling, every polygon we tried from $n = 200$

upward falls short of the 0.99 gate on exact coverage, reaching only 0.77–0.91 by $n \geq 800$. At the sizes in Table 12, then, disc-vis greedy is not a cheaper route to the same guard set; it cannot reach the gate at any guard count, and its timing is that of a method that stops short. Short of the ceiling the estimate is optimistic rather than merely noisy: points are drawn uniformly, so for a *fixed* guard set the estimate would be unbiased, but the algorithm reports the maximum it selected on, so the gap is positive on every polygon we measured and tracks the number of rounds rather than n (Table 13).[3] Training is less exposed, because the PO/BT reward averages one noisy sample per rollout over many rollouts and polygons and the editor refines an already-good seed rather than building one from empty over dozens of rounds (Section 8), which is why we use the approximation for training and local-search targets but always re-score with exact CGAL at evaluation.

On memory, the disc-vis matrix is $n \times 500$ booleans, about a megabyte at the top of the size grid, while the learned pipeline is a fixed ≈ 0.83M parameters independent of n, plus $O(nd)$ activations; peak process memory was comparable across n for both classical implementations, dominated by fixed library overhead rather than by the growing exact polygon-set representation. We report the cost profile because it is what the geo-free constraint changes, not as a case for deploying the learned pipeline: classical greedy wins on guard count regardless of which scoring rule it uses (Section 6.1).

[3] Over a family-diverse sample of 40 polygons spanning $n \in [20, 1750]$, the Spearman correlation of the gap with guard count is 0.95 against 0.88 with n, and controlling for the other separates them (0.78 partial correlation for guard count, 0.29 for n). Two polygons at the same $n = 500$ make the point directly: one needing 3 guards has a gap of $+0.004$, one needing 56 has a gap of $+0.044$.

	Exact greedy		Disc-vis greedy		Geo-free learned		
n	precompute (ms)	selection (s)	precompute (ms)	selection (ms)	precompute	CPU (ms)	GPU (ms)
200	119	1.2	338	1.5	—	12	6
500	286	6.3	932	5.9	—	33	13
1000	620	20.4	1664	19.8	—	89	25
2000	1609	83.6	4196	77.9	—	264	57

Table 12: Per-instance inference cost, split by mechanism rather than by method: *precompute* builds a structure once; *selection* then produces the guard set, querying that structure once per candidate over $\approx n/6$ rounds for the two classical methods. That query is expensive for exact greedy (an exact CGAL polygon-set union per candidate) and cheap for disc-vis greedy (one vectorized lookup against the precomputed $M = 500$-sample matrix). The geo-free pipeline has nothing to precompute, since it never reads a visibility structure at inference (Section 2.3); its CPU and GPU columns are each a single coordinate-only forward pass standing in for selection. Exact greedy's selection is given in seconds, unlike the other columns, since it runs orders of magnitude above them; its growth is discussed in the text.

n	guards	disc-vis coverage (est.)	exact coverage (true)	gap
200	20.5 ± 7.3	0.991	0.973 ± 0.007	0.018 ± 0.006
500	38.2 ± 18.6	0.991	0.950 ± 0.016	0.041 ± 0.015
1000	75.3 ± 9.0	0.990	0.907 ± 0.010	0.083 ± 0.010
2000	139.0 ± 10.4	0.990	0.807 ± 0.006	0.183 ± 0.006

Table 13: Disc-vis greedy's stopping quality, mean $\pm$ std over 8/5/3/3 polygons per bucket. "disc-vis coverage" is the sample estimate it stops on (target 0.99); "exact coverage" is the same guard set scored by CGAL afterwards.

7. Discussion

The central measurement in this paper is what happens when we read the encoder rather than the decoder. Reading only the frozen embeddings, the coordinates, and the policy's seed indicator, with no visibility matrix or CGAL output, the SetPredictor shortens the in-distribution coverage tail by at least four-fold on every probe seed (Table 2) and cuts the out-of-distribution one by about an order of magnitude (Section 6.3), on all four policies we trained (Section 6.4). Two readings fit this. Either the encoder learned enough geometry that a small classifier over its output recovers coverage, or the decoder's EOS calibration was the bottleneck and the policy had already found the right vertices but stopped early. Both give the same headline: this policy internalized a representation holding more of the geometry than its decoder expressed. Four measurements favour the encoder reading (Section 6.2): at matched guard budget the no-encoder ablation must spend appreciably more guards to leave the same tail, and coordinates alone more still; masking only the seed barely moves the probe, so the seed does not carry the signal; a linear readout of the frozen embedding already ranks guards above non-guards (Table 7), so the structure is accessible through a modest readout rather than built by the probe; and the feature-set control (Table 7) shows that structure is not reducible to hand-designed geometry, analytic features recover much of the gain, yet appending them to the embedding still improves the readout in every fold, so the encoder carries guard-relevant information cheap geometry does not, and vice versa.

Two caveats bound the claim. Successful probing shows the LS target is

decodable from the features, which is necessary for the encoder to carry the geometry but does not prove it carries all of it, nor that the residual failures are solely decoder calibration; we give the encoder reading as the better-supported one, not the only one. And the word *geometry* needs qualifying, because geometry is invariant to rigid motions and this representation is not: rotating or reflecting a polygon costs a large fraction of the gate-clearing count (Section 8), and the feature-set control locates the same deficit mechanistically, appending the interior angle, a rigid-motion invariant, to the embedding improves the readout, so the encoder demonstrably does not carry it. What the probe reads is therefore guard-relevant structure expressed in the canonical presentation of the training ecosystem, enough to recover the coverage decision on instances written the same way, but not the orientation-independent notion of visibility a geometer would mean. We take this as a statement about what *this* training setup produces rather than a limit on what encoder probing can reveal.

Recovering coverage has a price, and we report it rather than disguise it. As the threshold rises the probe's guard cost falls the whole way from well above the optimum to the policy's own cardinality (Figure 3), and the $|S|/\mathrm{OPT}$ box plots of Figure 4 show that the whole distribution widens as the threshold falls, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

That price is not incidental, and reading it together with the capacity ladder suggests where the limit actually lies. Coverage is submodular, so a vertex is worth what the already-chosen guards leave uncovered, and an autoregressive pointer can condition on that partial solution (Section 4.1), which is why the policy attains near-classical cardinality. The probe cannot, because it emits one probability per vertex and thresholds them independently. It can therefore rank vertices by guard-relevance but cannot reason about marginal gain, and the only way an independent rule can guarantee coverage is to select past the point where the marginal vertex is needed. The over-guarding is the signature of that constraint rather than a tuning artifact. Two observations line up with this reading: the probe selects roughly 1.8× the cardinality of the local-search sets it is trained on (Table 2) while still ranking their members above non-members at ROC-AUC 0.923, and the capacity ladder shows self-attention adding almost nothing over an attention-free readout (Table 6), if the bottleneck were representational, attention over vertices is exactly the capacity that should have helped. We therefore read the evidence as showing that the frozen encoder carries guard-relevant structure that a modest readout recovers, while converting that structure into a *minimal* covering set needs the sequential conditioning the single-shot probe gives up. This is a statement about what a per-vertex readout can express, not a claim that the encoder lacks the remaining information, and it is the form of the limit our experiments can actually speak to.

Note that we are not claiming a better AGP solver. Classical greedy reaches near-full coverage at close to the optimal guard count on `test` (Table 2); the probe at $t = 0.20$ reaches comparable coverage but at about 1.6× the optimum across four seeds. A greedy-then-local-search pipeline, the standard high-quality

classical recipe, would be stronger still on cardinality; the *LS on policy seed* row of Table 2, which refines a guard set with the same add/remove/swap editor to clear the 0.95 gate on all 362 at lower cardinality still, already bounds what such refinement achieves on this data. The policy and probe are not built to contest that. The contrast between the failed iterative editors (Section 6.5) and the single-shot probe is the cleanest expression of what we do document: removing the stop time and the rollout is exactly what lets the encoder's information surface.

8. Limitations

We acknowledge several limitations of the work, enumerated here for clarity; the first two are conceptual and the rest experimental.

1. **Scope of the claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time, whereas *training* uses discretized visibility for both the PO/BT reward and the probe's supervised targets (see the approximation item below), so "no explicit geometric knowledge" must be read as "no geometric oracle at the moment of placing guards." The SetPredictor is also supervised rather than reinforcement-trained, by BCE against local-search labels; we position it as a representation probe of the RL-trained encoder, not as an RL contribution.
2. **Instance ecosystem and evaluation size.** The in-distribution held-out set is modest (Wilson 95% intervals are reported throughout, but `test` is a few hundred polygons) and `ood` is the larger second evidence base (Table 1). More importantly, although the evaluation spans several polygon families and a wide size range, nearly all instances originate from one ecosystem (the AGPVG library [18] plus our same-format `random` supplement), so the generator and coordinate format are largely shared between training and evaluation. The out-of-distribution axis we stress is therefore predominantly *size* extrapolation, not a shift to a genuinely different polygon distribution or instance source. Whether the encoder's geometry transfers across instance ecosystems (procedurally different generators, or real floorplans) is untested and left to future work.
3. **Policy selection and seeds.** All results outside Section 6.4 come from one released policy, the best of several development runs by greedy-decoded `train` performance; that choice and the within-run checkpoint rule read only `train` (Section 5.2). Selecting on training reward rather than a held-out criterion is a weaker rule that, if anything, risks favoring an over-fit run, so we make no claim the selected policy is optimal, and it sits at the favourable end of the runs we trained. The three replication policies show that both load-bearing results survive a change of policy, but two gaps remain: we did not sweep the full threshold range out of distribution for them, so the matched-budget comparison on `ood` and `ood-large` rests on

the released policy alone, and the no-seed and coordinates-only conditions exist only for it.

One replication run shows how much variance the recipe carries, and why the within-run criterion matters. Policy seed 22 collapsed onto over-guarding: its greedy decode reaches an estimated coverage of 1.000 while selecting almost every vertex, $|S|/n$ staying above 0.92 from the second epoch until the run early-stopped at epoch 41. That endpoint scores -0.008 under Eq. (3), against $+0.736$ and $+0.737$ for the two seeds that trained normally, so it failed to find a good optimum rather than gaming the reward. It is also the failure a raw-coverage checkpoint rule cannot see: coverage alone peaks at epoch 8, covering everything with 96% of the vertices, whereas subtracting the guard fraction (Section 5.2) rejects that point. The same policy floods guards at evaluation on `ood-large`, which is why that cell is excluded from the encoder comparison (Section 6.4).

4. **Design choices we did not sweep.** Three components were fixed at the values that produced the released checkpoint rather than searched. A Transformer encoder–decoder policy of roughly $6\times$ the parameters reached a comparable coverage–cost operating point, with one run failing to converge; we read the absence of gain as overparametrization relative to the scalar RL signal rather than as evidence against attention models, since the probe is itself a small Transformer and works on dense per-vertex labels. Decode-time search by best-of-K sampling matched greedy and did not close the tail (Section 6.5); we did not implement beam search or symmetry-exploiting RL such as POMO [16], both of which act on the decoder rather than on what the encoder has learned. The reward weights and PO settings (Section 5.2) were not varied: the design turns on the *form* of the reward, capping coverage at τ and penalizing the deficit linearly, not on the precise scalars, and a sensitivity study would mean re-running PO/BT training. All three are left to future work.
5. **Discretized-visibility approximation in training.** We did not sweep $M = 500$ (Section 4.1), but we did measure what the approximation costs the training signal, since PO/BT consumes the *ordering* of two rollouts rather than their reward magnitudes. Re-scoring 1600 rollouts of the frozen policy under both estimators, the discretized coverage is unbiased against exact CGAL (mean difference $+0.0003$, optimistic on 52% of rollouts) with no trend in polygon size or guard count, and the induced pairwise preferences agree on 91.9% of pairs. The disagreements sit on near-ties: agreement is 100% once the exact reward gap exceeds 0.10, so the flipped pairs carry almost no preference signal. One asymmetry does survive: because most rollouts sit just below the coverage gate, symmetric sampling noise crosses it upward more often than downward (94 rollouts credited with clearing τ that exact scoring places below it, against 22 the other way), which mildly rewards under-covering and may contribute to the residual tail. A larger M, or stratified sampling near reflex vertices, would shrink that effect; quantifying the full sensitivity is left to future work.

6. **Extreme OOD: the encoder signal is confined to the worst case.** On `ood-large` the encoder's advantage shows up in worst-case coverage and nowhere else (Section 6.3), which we read as the signal persisting at eleven times the training size rather than as a coverage guarantee at that scale. The classical references were not computed for this split and the optimum is unavailable for its largest polygons, so guard cost here can be compared only against the policy's own seed.
7. **The pipeline is *not* invariant to rotation or reflection; vertex-order dependence is removable.** Normalization handles translation and axis-aligned scaling by construction (Section 5.2), but nothing enforces the other symmetries, and measurement shows they are not acquired. We re-ran the frozen policy and probe ($t = 0.20$) on transformed `test` polygons (rotations of 45°, 90° and 180°, a mirroring, and a cyclic re-index) re-decoding the seed and re-normalizing each time so the model sees a valid input. Every transformation costs a large fraction of the result: the gate-clearing count falls from 345/362 under identity to between 205 and 312, and the worst polygon from 0.857 to 0.382. The rotations are clean tests of congruence, since re-normalizing a rotated unit square is isotropic and only the presentation to the encoder changes; the mirroring row is confounded, because obtaining a validly oriented reflected polygon also reverses the vertex sequence. The effect is not a property of the probe, the seed alone degrades under the same rotations, so the orientation dependence sits in the encoder, and the probing result should be read as a statement about representations learned under this ecosystem's conventions rather than about orientation-independent geometry.
   The two failure modes are not equally serious, because one is removable at no cost: canonicalizing the traversal so that a geometrically determined vertex comes first makes any cyclic re-index exactly invariant by construction. Which vertex to use matters, so we selected the rule on the disjoint `dev` split, leaving the reporting split untouched; the chosen rule, the lexicographic maximum in (x, y), matches native file order on `test` (346/362 against 345/362) while making the output identical under every cyclic roll, whereas our first guess, the lexicographic *minimum*, was catastrophic, which is why the rule has to be selected rather than assumed. Rotation is not helped, since the canonical vertex is not itself rotation-equivariant. Orientation dependence therefore remains the substantive open item, and the natural remedies are architectural: rigid-motion-invariant inputs such as per-vertex edge lengths and turning angles (which determine a polygon up to rigid motion, and lose nothing, since visibility is itself a rigid-motion invariant), a rotation-equivariant encoder, or orientation augmentation during PO/BT training. The present measurements are what make that a testable hypothesis rather than a guess.

9. Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder. A single-shot probe over the frozen embeddings shortens the in-distribution coverage tail several-fold and compresses the out-of-distribution one by roughly an order of magnitude across four probe seeds, on all four independently trained policies we tested. We take this as evidence that the reinforcement-trained encoder had captured more of the placement structure than its decoder emitted as guards. That structure is tied to the presentation it was trained on, it does not survive rotation or reflection, so the claim is about what this training setup encodes, not about an orientation-independent representation of visibility.

Three extensions follow. The most promising, given that orientation dependence, is an encoder invariant to rigid motions, along the architectural lines of Section 8, where we also show the companion vertex-order dependence needs no such change. A stronger encoder such as a Transformer trained under the same PO/BT objective may shorten the residual tail directly and leave less work for the probe. And a per-instance threshold, predicted geo-free from the same embeddings but trained against coverage-selected targets exactly as the probe is trained against the visibility-derived labels $S^{\star}_{\mathrm{LS}}$, would replace the global knob with an adaptive one, keeping visibility a training-time target and never an inference-time input.

More broadly, the encoder-probing decomposition need not be specific to vertex guarding: we conjecture it transfers to other geometric set-cover problems whose coverage oracle is costly at inference, such as terrain guarding or range-limited watchtower placement. Whether a frozen encoder carries the requisite structure there is an open question this study cannot answer on its own, our conclusions concern the PO/BT pointer architecture, replicated across four training runs of it, and establishing that they hold across architectures and learning rules would require probing more than one kind of solver. The learner is not the best AGP heuristic, but it arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference.

Data and Code Availability

The AGPVG instance library is publicly available [18]. The 313 supplemental `random`-class polygons with their ILP-computed vertex-guard optima, the dataset splits, the trained checkpoints, and all training and evaluation code are available at `https://github.com/dseverdi/AGNet`.

Conflict of Interest

The authors declare no competing interests.

References

[1] D. T. Lee, A. K. Lin, Computational complexity of art gallery problems, IEEE Transactions on Information Theory 32 (2) (1986) 276–282.

[2] J. O'Rourke, Art Gallery Theorems and Algorithms, Oxford University Press, 1987.

[3] A. Elnagar, L. Lulu, An art gallery-based approach to autonomous robot motion planning in global environments, in: 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2005, pp. 2079–2084. doi:10.1109/IROS.2005.1545170.

[4] B. S. Bigham, S. Dolatikalan, A. Khastan, Minimum landmarks for robot localization in orthogonal environments, Evol. Intell. 15 (3) (2022) 2235–2238. doi:10.1007/s12065-021-00616-8.
URL https://doi.org/10.1007/s12065-021-00616-8

[5] M. Pan, G. Lin, Y.-W. Luo, B. Zhu, Z. Dai, L. Sun, C. Yuan, Preference optimization for combinatorial optimization problems, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2505.08735.

[6] R. A. Bradley, M. E. Terry, Rank analysis of incomplete block designs: I. the method of paired comparisons, Biometrika 39 (3–4) (1952) 324–345.

[7] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Machine Learning 8 (3-4) (1992) 229–256.

[8] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio, Neural combinatorial optimization with reinforcement learning, arXiv preprint arXiv:1611.09940 (2016).

[9] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes (2017). arXiv:1610.01644.

[10] J. Hewitt, P. Liang, Designing and interpreting probes with control tasks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 2733–2743.

[11] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Computational Linguistics 48 (1) (2022) 207–219.

[12] Z. Zhang, Y. Ma, Z. Cao, H. C. Lau, Probing neural combinatorial optimization models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 38, 2025, spotlight; arXiv:2510.22131; extended version at OpenReview forum:agEy9hliY1, "Unveiling Neural Combinatorial Optimization Model Representations through Probing".

[13] R. Narad, L. Boussioux, M. Wagner, Probing neural TSP representations for prescriptive decision support (2026). arXiv:2602.07216.

[14] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 28, 2015.

[15] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR), 2019.

[16] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min, POMO: Policy optimization with multiple optima for reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020.

[17] M. C. Couto, P. J. de Rezende, C. C. de Souza, An exact algorithm for minimizing vertex guards on art galleries, International Transactions in Operational Research 18 (4) (2011) 425–448.

[18] M. C. Couto, P. J. de Rezende, C. C. de Souza, Instances for the Art Gallery Problem, http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery (2009).

[19] S. K. Ghosh, Approximation algorithms for art gallery problems in polygons, Discrete Applied Mathematics 158 (6) (2010) 718–722.

[20] H. González-Baños, A randomized art-gallery algorithm for sensor placement, Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG) (2001) 232–240.

[21] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, An analysis of approximations for maximizing submodular set functions—i, Mathematical Programming 14 (1) (1978) 265–294.

[22] B. Ben-Moshe, M. J. Katz, J. S. B. Mitchell, A constant-factor approximation algorithm for optimal 1.5D terrain guarding, SIAM Journal on Computing 36 (6) (2007) 1631–1647.

[23] K. Elbassioni, D. Matijević, J. Mestre, D. Ševerdija, Improved approximations for guarding 1.5-dimensional terrains, CoRR abs/0809.0159v1 (2008).

[24] Y.-H. Liao, P.-C. Chang, H.-H. Li, Reinforcement learning for optimize coverage in art gallery problem using Q-learning based in grid world, IEEE Access 13 (2025) 52711–52724.

[25] M. D. Kaba, M. G. Uzunbas, S. N. Lim, A reinforcement learning approach to the view planning problem (2016). arXiv:1610.06204.

[26] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, L.-M. Rousseau, Learning heuristics for the TSP by policy gradient, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR), Vol. 10848 of Lecture Notes in Computer Science, Springer, 2018, pp. 170–181.

[27] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, J. Bian, Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem, aAAI 2023 (2023). arXiv:2304.09407.

[28] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. doi:https://doi.org/10.1016/j.ejor.2020.07.063.
URL https://www.sciencedirect.com/science/article/pii/S0377221720306895

[29] I. Mehta, S. Taghipour, S. Saeedi, Pareto frontier approximation network (PA-Net) to solve bi-objective TSP (2022). arXiv:2203.01298.

[30] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, IEEE Transactions on Cybernetics 51 (6) (2021) 3103–3114.

[31] C. Caramanis, D. Fotakis, A. Kalavasis, V. Kontonis, C. Tzamos, Optimizing solution-samplers for combinatorial problems: The landscape of policy-gradient methods, neurIPS 2023 (2023). arXiv:2310.05309.

[32] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011, pp. 627–635.

[33] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.

[34] The CGAL Project, CGAL user and reference manual, https://doc.cgal.org/5.6/Manual/packages.html (2024).

[35] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.